

附录 B: Solutions to End-of-Part Exercises (各部分练习解答)

手机

附录 B: Solutions to End-of-Part Exercises (各部分练习解答)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本附录地位：全书"官方答案手册"——收录 Part I 到 Part VII 各部分章末练习（Test Your Knowledge: Part Exercises）的参考解答。它不教新语法，而是把前面七大部分学过的所有知识点（对象类型、语句、函数、模块、类、异常）串成可运行的代码，演示"标准用法 + 常见坑"，并配大量真实文件（module1.py、adder2.py、primes.py、mylist.py、zoo.py、exctools.py 等）。重要说明：解答中由标题或叙述引用的代码文件，均位于书籍示例包（examples package）的 AppendixB 文件夹中，该文件夹为每个部分各设一个子文件夹（例如 AppendixB/Part1 存放第一部分的文件）。关于示例包的更多信息，请参阅《前言》（Preface）。

引言 (Introduction)

英文原文

This appendix provides solutions for the book's end-of-part exercises. Code files named by captions or na

rative in these solutions are available in the book examples package's AppendixB folder, which has one subfolder per part (e.g., AppendixB/Part1 is the first part's files). See the Preface for more info on the examples package.

中文翻译

本附录为全书各部分章末练习提供官方解答。解答中由标题 (captions) 或叙述 (narrative) 所引用的代码文件, 都存放在本书示例包 (examples package) 的 AppendixB 文件夹中, 该文件夹为每个部分各设一个子文件夹 (例如 AppendixB/Part1 就是第一部分的文件)。关于示例包的更多信息, 请参阅《前言》。

深度理解

- 核心概念: 附录 B 是"练习答案库", 但作者的设计意图是先做题、后对照。每题答案都展示了"官方推荐的解题思路", 而非唯一答案。
- 代码文件与书页的对应: 正文中的 Example B-N (示例 B-数字) 编号对应本附录, 文件路径如 Part4/adder2.py 指示例包内 AppendixB/Part4/adder2.py。
- 学习建议: 逐题对照时, 重点关注三件事——① 为什么这么写; ② 输出结果的底层原因; ③ 自己写法的差异点在哪。

Part I: Getting Started (入门)

英文原文

Part I, Getting Started See "Test Your Knowledge: Part I Exercises" in Chapter 3 for the exercises.

中文翻译

第一部分：入门。练习见第 3 章的 "Test Your Knowledge: Part I Exercises"（第 I 部分练习测验）。

深度理解

- 核心概念：Part I 的练习全部围绕"怎么让 Python 跑起来"——交互式解释器、脚本文件、模块导入、shebang 启动、错误消息、循环引用对象。
- 底层视角：这部分在帮你建立两个心智模型：① REPL（交互模式）与 .py 文件的差异；② 模块执行与字节码缓存（.pyc）的关系。
- 常见误区：初学者常以为 import 是"复制粘贴代码"，其实它是"执行一次模块文件"，所以重复 import 不会重复执行，除非显式 reload。

题 1. Interaction (交互式会话)

英文原文

1. Interaction: Assuming Python is configured properly, the interaction should look something like the following. You can run this any way you like—in IDLE, a console, an app, a notebook's page, and so on:

```
$ python3
...information lines...
>>> 'Hello World!'
'Hello World!'
>>> # Use ctrl+D/ctrl+Z to exit on Unix/Windows, or close window
```

中文翻译

1. 交互式会话：假设 Python 已正确配置，交互过程看起来应大致如下。你可以用任何喜欢的方式运行它——在 IDLE、控制台、某个应用（app）、notebook 页面等等：

```
$ python3
.....
>>> 'Hello World!'
'Hello World!'
>>> # Unix/Windows ctrl+D/ctrl+Z
```

代码分析

```
$ python3          # REPL--
>>> 'Hello World!' #
'Hello World!'    # echo— repr
>>>               # ctrl+D(Unix)/ctrl+Z(Windows)
```

深度理解

·核心概念：REPL 是 Python 的"活体实验台"。每个输入

表达式都会经过编译为字节码 → 解释求值 → 打印结果三步。

·底层机制：交互模式下解释器会对每个完整表达式自动调用 `repr()` 打印结果（字符串带引号、列表带方括号）；而脚本文件里不会自动回显——这正是“交互”与“脚本”最重要的差别，后面 Part IV 练习 2 还会专门强调。

·为什么这样设计：让新手零成本感受“无编译步骤”的开发速度，写完即运行。

·常见误区：把 REPL 里能自动看到结果，误以为在文件里写 `x + 1` 也会输出——不会，必须 `print`。

题 2. Programs (程序)

英文原文

2. Programs: Your code (i.e., module) file should look something like Example B-1: Example B-1. Part1/module1.py `python print('Hello module world!')` And here is the sort of interaction you should have; for console launches, be sure to use your platform's version of the "python3" command (e.g., try "py -3" on Windows):
`$ python3 module1.py` Hello module world! Again, feel free to run this other ways—by clicking or tapping the file's icon, by using IDLE's Run→Run Module menu option, by UI options in web notebooks or other IDEs, and so on.

中文翻译

2. 程序：你的代码（也就是模块）文件应该像示例 B-1 这样：`python print('Hello module world!')` 你应该得到的交互输出如下；从控制台启动时，务必使用你平台对应的 "`python3`" 命令（例如 Windows 上尝试 "`py -3`"）：`$ python3 module1.py` Hello module world! 同样，你也可以用其他方式运行它——点击/轻点文件图标、使用 IDLE 的 `Run→Run Module` 菜单选项、web notebook 或其他 IDE 的 UI 选项等等。

代码分析

```
print('Hello module world!')    # print
```

深度理解

- 核心概念：一个 `.py` 文本文件就是一个模块（module），也就是一个程序。文件被读入内存、编译成字节码、交给解释器虚拟机执行。
- 底层机制：`python3 module1.py` 底层就是“把该文件当作名为 `main` 的模块导入执行”——所以文件里的 `print` 语句会在执行到的那一瞬间输出。
- 多种运行入口的等价性：IDLE 的 `Run→Run Module` 本质是“在解释器进程内 `import` 该文件”；双击图标则是让操作系统关联程序打开它。
- 常见误区：Windows 上 `python3` 命令可能不存在，需用 `py -3`；文件路径含空格（如 My Documents）时命令必须加引号。

题 3. Modules (模块)

英文原文

3. Modules: The following interaction listing illustrates running a module file by importing it: `$ python3 > > import module1` Hello module world! Remember that you will need to reload the module to run it again without stopping and restarting the interactive interpreter (i.e., REPL). Moving the .py file to a different directory and importing it normally fails: Python likely generated a module1.pyc file in the pycache subdirectory of the source code file's folder, but it won't use it when you import the module there if the source code (.py) file has been moved elsewhere and to a folder not in Python's import search path. The .pyc file is written automatically if Python has access to the source file's directory; it contains the compiled bytecode version of a module. See Chapter 3 for more on modules, Chapter 2 for more on bytecode, and Chapter 2 ahead for more on both. To really use the saved .pyc sans .py, as of Python 3.2, you must move it up one level and rename it without the "" part in the middle, or generate it from and alongside the source code file with the Python compileall module's "legacy" (-b) mode. For example, the following compiles all source code files in the current directory into directly usable bytecode files (you can also list specific files or recurs

e into subfolders, per Python library docs): `$ python3 -m compileall -b -l .`

中文翻译

3. 模块：下面的交互式列表演示如何通过导入 (import) 来运行一个模块文件： `$ python3 >>> import module1`
Hello module world! 记住：要想在不停止、不重启交互式解释器（即 REPL）的情况下再次运行模块，你需要 `reload`（重新加载）它。把 `.py` 文件移动到另一个目录后再正常导入会失败：Python 很可能已经在源文件所在文件夹的 `pycache` 子目录里生成了 `module1.pyc` 文件，但如果源文件 (`.py`) 已被移到别处、且不在 Python 的导入搜索路径 (import search path) 中，它就不会使用这份缓存。只要 Python 对源文件所在目录有写权限，`.pyc` 文件就会被自动写出；它包含模块编译后的字节码版本。关于模块参见第 3 章，关于字节码参见第 2 章，关于两者的更多内容参见后面的第 22 章。如果想真正在缺少 `.py` 的情况下使用保存的 `.pyc`，从 Python 3.2 起你必须把它向上移一级、去掉中间带的部分改名，或者用 Python `compileall` 模块的“旧式” (`-b`) 模式，从源文件生成并与之并存的字节码文件。例如，下面的命令把当前目录中的所有源代码文件编译成可直接使用的字节码文件（按 Python 库文档，你还可以列出具体文件或递归进入子文件夹）：`$ python3 -m compileall -b -l .`

代码分析

```
import module1          # module1.py
# Hello module world!
```

python3 -m compileall -b -l . 的含义：

- -m compileall：以"模块方式"运行内置的批量字节码编译器；
- -b (legacy 旧式)：把 .pyc 放在源文件旁边、且文件名不含版本号中间段（供 Python 3.2 及更早的"旁边式"读取）；
- -l：不递归子目录，只处理当前目录；
- .：当前目录（也可列出具体的文件路径）。

深度理解

- 核心概念：import 一个模块会执行整份文件，但只会执行一次——模块对象会被缓存进 sys.modules，第二次 import 直接返回缓存。
- 字节码与 .pyc 的关系：Python 源码先被编译成字节码 (bytecode) 再执行；.pyc 就是字节码的磁盘缓存。若 .py 的时间戳/校验和未变，则跳过编译直接加载 .pyc，启动更快。
- 为什么移动 .py 后 import 失败：import 靠搜索路径 (sys.path：当前目录、PYTHONPATH、标准库目录等) 找模块，.pyc 不会自动跟着源文件"搬家"。
- 设计思想：.pyc 是"以空间换时间"的产物；pycache 子目录是为了让多个 Python 版本、多种解释器 (CPython / PyPy) 的缓存文件共存互不干扰。
- 常见误区：手动删除 .pyc 不会让程序坏掉（最多重新编译）；在只读目录里运行不会生成 .pyc，这很正常，不是

错误。

题 4. Scripts (脚本)

英文原文

4. Scripts: Assuming your platform supports the `#!` trick, your solution will look like Example B-2, although your `#!` line may need to list a different path to Python on your machine. This line is significant under the Windows launcher shipped and installed with Python, where it is parsed to select a version of Python to run the script, despite the Unix path syntax, and subject to a default setting; see Appendix A and Python's docs for more details. This launching scheme is optional and generally less portable than others. Example B-2.

Part1/script1.py

```
python #!/usr/local/bin/python3 print('Hello module world!')
```

Running this as a program by console command and line:

```
$ chmod +x script1.py # See also: #!/usr/bin/env python3
$ ./script1.py # "./" needed only if "." not on PATH
Hello module world!
```

```
$ python3 script1.py # Or run normally and portably
Hello module world!
```

中文翻译

4. 脚本：假设你的平台支持 `#!` (shebang) 技巧，你的解决方案应该像示例 B-2 那样，不过你的 `#!` 行可能需要填写你机器上不同的 Python 路径。这一行在随 Python 安装的 Windows 启动器 (launcher) 下意义重大：尽管它写的是 Unix 路径语法，启动器会解析它来选择运行该脚本的 Python 版本，并且受默认设置约束；更多细节参见附录 A 和 Python 文档。这种启动方案是可选的，而且通常比别的方案可移植性差。

```
python #!/usr/local/bin/python3 print('Hello module world!')
```

通过控制台命令行把它当作程序运行：

```
$ chmod +x script1.py # 赋予可执行权限；另见：#!/usr/bin/env python3 $ ./script1.py # 仅当 "." 不在 PATH 上时才需要 "./" Hello module world! $ python3 script1.py # 或者正常、可移植地运行Hello module world!
```

代码分析

```
#!/usr/local/bin/python3    # Unix  shebang#!...  
print('Hello module world!')
```

- `chmod +x script1.py`：赋予文件可执行权限 (Unix/macOS 需要) 。
- `./script1.py`：显式指出“在当前目录下执行”——若 `.` 不在 `PATH` 环境变量中就必须加 `./`。
- `python3 script1.py`：绕过 shebang，直接指定解释器运行，最可移植。

深度理解

- 核心概念：shebang 让脚本可以像系统命令一样直接执行；操作系统看到 `#!` 行就调用指定解释器。
- Windows 上的特殊行为：Windows 的 `py` 启动器会解析 `#!` 行（尽管是 Unix 语法）来选择 Python 版本（如 `py -3`、`py -3.11`），受默认版本设置影响。
- 可移植性问题：不同机器上 `/usr/local/bin/python3` 未必存在；`#!/usr/bin/env python3` 通过 `PATH` 查找更通用，但依赖 `PATH` 配置。这就是作者说它“通常可移植性差”的原因。
- 常见误区：在 shell 里直接敲 `python3 script.py` 时 `shebang` 根本不会被读取；Windows 双击 `.py` 靠的是文件关联（file association），与 `shebang` 无关。

题 5. Errors and debugging（错误与调试）

英文原文

5. Errors and debugging: The following interaction demonstrates the sorts of error messages you'll get when you complete this exercise. Really, you're triggering Python exceptions; the default exception-handling behavior terminates the running Python program and prints an error message and stack trace on the screen. The stack trace shows where you were in a program when the exception occurred (if function calls are

active when the error happens, the "Traceback" section displays all active call levels).

In Chapter 10 and Part VII, you will learn that you can catch exceptions using try statements and process them arbitrarily. You'll also learn that Python includes a full-blown source code debugger (module pdb) for special error-detection requirements. For now, notice that Python gives meaningful messages when programming errors occur, instead of crashing silently:

```
$ python3 >> 2 500 32733906078961418700131896
96827599152216642046043064789483291368096133
7964 046745 548832700923259041571508866841275
60071009217256545885393053328527589376
```

```
>> 1 / 0 Traceback (most recent call last): File "<stdin>", line 1, in <module> ZeroDivisionError: division by zero
```

```
>> oops Traceback (most recent call last): File "<stdin>", line 1, in <module> NameError: name 'oops' is not defined
```

中文翻译

5. 错误与调试：下面的交互过程演示了你完成这个练习时会看到的各类错误消息。说真的，你是在触发 Python 的异常 (exception)；默认的异常处理行为会终止正在运行的 Python 程序，并在屏幕上打印一条错误消息和堆栈跟踪 (stack trace, 即回溯 traceback)。堆栈跟踪显示异常发

生时你在程序中的位置（如果出错时函数调用正处于活动状态，"Traceback"部分会显示所有活动调用层级）。在第 10 章和第七部分，你会学到用 try 语句捕获异常并按需处理；你还会学到 Python 内置了一个功能完整的源码级调试器（模块 pdb），用于特殊的错误检测需求。现在请先注意：发生编程错误时，Python 会给出有意义的提示消息，而不是悄悄崩溃：

```
$ python3 >>> 2 500 3273390607896
14187001318969682759915221664204604306478948
32913680961337964 046745 54883270092325904157
15088668412756007100921725654588539305332852
7589376 >>> 1 / 0
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> oops
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'oops' is not defined
```

代码分析

```
>>> 2 ** 500          # Python int ""2 500
>>> 1 / 0             # →
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>    # <stdin> 1 ...
ZeroDivisionError: division by zero    # +

>>> oops              # → NameError
NameError: name 'oops' is not defined
```

深度理解

·核心概念：错误分两类——语法错误（编译期发现）与运行时异常（执行期抛出）。这里的 1/0、oops 都是运行时异常。

· Traceback 的读法：从下往上读！最下面一行是"异常类型: 消息"，往上每层 File "...", line N, in 函数名显示异常冒泡经过的调用链。

· 底层机制：异常是对象，由解释器 raise 抛出，沿调用栈向上传播，最终被默认处理器捕获并打印、终止程序。pdb 调试器可在异常处断下，try 语句则能捕获并转换处理（Part VII 详述）。

· 为什么这样设计：把运行期问题"显式化"，而不是像 C 语言那样直接段错误（segmentation fault）挂掉——这让新手能定位错误、让程序能优雅降级。

· 常见误区：新手看 Traceback 盯着最上面的 File "<stdin>" 一行，其实根源在最后一行的异常类型与消息。

题 6. Breaks and cycles（中断与循环）

英文原文

6. Breaks and cycles: When you type this code: `$ python3 >>> L = [1, 2] >>> L.append(L) >>> L [1, 2, [...]]` you create a cyclic data structure in Python. In Python releases before 1.5.1, the Python printer wasn't smart enough to detect cycles in objects, and it would print an unending stream of `[1, 2, [1, 2, [1, 2, [1, 2, and so on until you hit the Ctrl+C break-key combination on your machine (which, technically, raises a keyboard-interrupt exception that prints a default message). Beginning with Python 1.5.1, the printer is clever eno`

ough to detect cycles, prints [...] instead to let you know that it has detected a loop in the object's structure, and avoids getting stuck printing forever. The reason for the cycle is subtle and requires information you will glean in Part II, so this is something of a preview. But in short, assignments in Python always generate references to objects, not copies of them. You can think of objects as chunks of memory and of references as implicitly followed pointers. When you run the first assignment in the preceding code, the name `L` becomes a named reference to a two-item list object—a pointer to a piece of memory. Python lists are really arrays of object references, with an `append` method that changes the array in place by tacking on another object reference at the end. Here, the `append` call adds a reference to the front of `L` at the end of `L`, which leads to the cycle illustrated in Figure B-1: a pointer at the end of the list that points back to the front of the list. Figure B-1. A cyclic object, created by appending a list to itself. Besides being printed specially, as you'll learn in Chapter 6, cyclic objects must also be handled specially by Python's garbage collector, or their space will remain unreclaimed even when they are no longer in use. Though rare in practice, in some programs that traverse arbitrary objects or structures, you might have to detect such cycles yourself by keeping track of where you've been to avoid looping. Believe it or not, cyclic data structures can sometimes be use

ful, despite their special-case printing.

中文翻译

6. 中断与循环：当你输入这段代码时：`>>> L = [1, 2] >>> L.append(L) >>> L` `[1, 2, [...]]` 你就在 Python 中创建了一个循环（cyclic）数据结构。在 Python 1.5.1 之前的版本中，Python 的打印器不够聪明，无法检测对象中的循环，于是会一直打印 `[1, 2, [1, 2, [1, 2, [1, 2.....` 没完没了，直到你在机器上按下 `Ctrl+C` 中断键组合（从技术上讲，这会抛出一个打印默认消息的键盘中断异常）。从 Python 1.5.1 开始，打印器变得足够聪明，能检测循环，改而打印 `[..]`，让你知道它检测到了对象结构中的环，从而避免永远卡在打印上。循环产生的原因很微妙，需要用到你在第二部分获得的知识，所以这算是一种预告。简单来说：Python 中的赋值总是生成对对象的引用（references），而不是对象的副本。你可以把对象想象成内存块，把引用想象成隐式跟随的指针。运行前面代码中第一条赋值后，名字 `L` 成为一个对“双元素列表对象”的命名引用——一块内存的指针。Python 列表本质是对象引用的数组，`append` 方法通过在末尾再挂接一个对象引用来就地修改数组。这里 `append` 调用在 `L` 的末尾添加了一个指向 `L` 开头的引用，于是产生了图 B-1 所示的循环：列表尾部的一个指针，指回列表头部。除了被特殊打印之外，正如你在第 6 章会学到的，循环对象还必须由 Python 的垃圾回收器（garbage collector）特殊处理，否则即使它们不再被使用，其空间也无法回收。虽然实践中少见，但在一些遍历任意对象或结构的程序中，你可能需要自己通过记录“去过哪里”来检测这类循环，以避免死循环。信不信由你，循环数据结构有时还真有点用处

——尽管它们打印时是特殊格式。

代码分析

```
L = [1, 2]          # L
L.append(L)         # append L "L "
L                   # L = [1, 2, L]
# [1, 2, [...]]
```

图 B-1：一个循环对象——列表向自身追加自身而创建：列表尾部有一个指回列表头部的指针。

深度理解

·核心概念：变量名是引用而非“盒子”；赋值复制的是引用（指针），不是对象内容。这几乎是全书最重要的心智模型。

·为什么会产生环：列表是“引用数组”；L.append(L) 把 L 所指对象的引用追加进该对象自身，内部指针指向自己，形成环。

·打印的演变：Python 1.5.1 前打印器会无限递归打印；此后检测到环就打印 [...] 占位。同理，序列化（pickle）、遍历等库也会对环做特殊标记。

·垃圾回收的联动：CPython 用引用计数做主回收；但循环引用使计数永不为零，必须靠分代循环垃圾回收器（gc 模块）定期扫描检测回收。

·实际意义：自引用结构（如循环链表、图的缓存环）偶尔有用；但遍历任意对象时必须记录已访问对象，否则死循环。

·常见误区：L.append(L) 不是把 [1,2] 复制一份再放进列表，而是放进一个指向同一个对象的引用——对象只有一份。

Part II: Objects and Operations (对象与运算)

英文原文

Part II, Objects and Operations See "Test Your Knowledge: Part II Exercises" in Chapter 9 for the exercises.

中文翻译

第二部分：对象与运算。练习见第 9 章的 "Test Your Knowledge: Part II Exercises"。

深度理解

·核心概念：Part II 的练习覆盖数字、字符串、元组、列表、字典、文件等核心对象类型的索引、切片、拼接、方法调用、可变性、嵌套操作。

·学习方法：这些练习的最佳做法是自己在 REPL 里敲一遍——本部分答案全是交互会话，照敲一遍比背结论有效得多。

题 1. The basics (基础)

英文原文

1. The basics: Here are the sorts of results you should get, along with a few comments about their meaning. Again, note that `;` is used in a few of these to squeeze more than one statement onto a single line (the `;` is a statement separator), and commas build up tuples displayed in parentheses. See file `Part2/basics.txt` for copy/paste sans media, though typing these manually is a good way to practice syntax:

```
$ python3 # Numbers >> 2 16 65536 >> 2 / 5, 2 / 5.  
0 (0.4, 0.4)
```

```
# Strings >> 'hack' + 'code' 'hackcode' >> S = 'Python'  
>> 'grok' + S 'grok Python' >> S 5 'PythonPythonPythonPythonPythonPython'  
>> S[0], S[:0], S[1:] ('P', '', 'ython') >> how = 'fun' >> 'coding %s is %s!' % (S, how) 'coding Python is fun!'
```

```
>> 'coding {} is {}!'.format(S, how) 'coding Python is fun!'
```

```
>> f'coding {S} is {how}!' 'coding Python is fun!'
```

```
# Tuples >> ('x',)[0] 'x' >> ('x','y')[1] 'y'
```

```
# Lists >> L = [1, 2, 3] + [4, 5, 6] >> L, L[:], L[:0], L[-2],  
L[-2:] ([1, 2, 3, 4, 5, 6], [1, 2, 3, 4, 5, 6], [], 5, [5, 6]) >> (  
[1, 2, 3] + [4, 5, 6])[2:4] [3, 4] >> [L[2], L[3]] [3, 4] >>
```

```
L.reverse(); L [6, 5, 4, 3, 2, 1] >> L.sort(); L [1, 2, 3, 4, 5, 6] >> L.index(4) 3
```

```
# Dictionaries >> {'a': 1, 'b': 2}['b'] 2 >> D = {'x': 1, 'y': 2, 'z': 3} >> D['w'] = 0 >> D['x'] + D['w'] 1 >> D[(1, 2, 3)] = 4 >> D {'x': 1, 'y': 2, 'z': 3, 'w': 0, (1, 2, 3): 4} >> list(D.keys()), list(D.values()), (1, 2, 3) in D ([ 'x', 'y', 'z', 'w', (1, 2, 3)], [1, 2, 3, 0, 4], True)
```

```
# Empties >> [], [], (), {}, None ([[]], [ ], [], (), {}, None])
```

中文翻译

1. 基础：下面是你应得到的各类结果，附带少量关于其含义的说明。再次注意：其中有几处使用了；把多条语句挤到一行（；是语句分隔符），逗号会构成元组并以括号形式显示。需要“无媒体”（纯文本）的复制/粘贴版本，参见文件 Part2/basics.txt；不过手动敲这些是练习语法的好方法：

```
# 数字>>> 2 16 # 2 的 16 次幂65536 >>> 2 / 5, 2 / 5.0 # 真除法，保留余数（小数部分）(0.4, 0.4) # 字符串>>> 'hack' + 'code' # 拼接 (Concatenation) 'hackcode'>>> S = 'Python'>>> 'grok' + S 'grok Python'>>> S 5 # 重复 (Repetition) 'PythonPythonPythonPythonPython'>>> S[0], S[:0], S[1:] # 索引、空切片、子串('P', '', 'ython') # S[:0] 是开头处的空切片 [0:0]，返回同类型的空值>>> how = 'fun'>>> 'coding %s is %s!' % (S, how) # 百分号格式化 (expression) 'coding Python is fun!'>>> 'coding {} is {}'.format(S, how) # format 方法'coding Pyt
```

```

hon is fun!>>> f'coding {S} is {how}!' # f-string 字面
量'coding Python is fun!' # 元组>>> ('x,')[0] # 索引单元
素元组'x'>>> ('x','y')[1] # 索引双元素元组'y'# 列表>>>
L = [1, 2, 3] + [4, 5, 6] # 列表拼接>>> L, L[:], L[:0], L[-2
], L[-2:] # 列表操作: 整表/切片/空切片/负索引/负切片([1,
2, 3, 4, 5, 6], [1, 2, 3, 4, 5, 6], [], 5, [5, 6]) >>> ([1, 2, 3]
+ [4, 5, 6])[2:4] # 拼接后再切片[3, 4] >>> [L[2], L[3]] #
从偏移位置取值, 存入新列表[3, 4] >>> L.reverse(); L #
方法: 就地反转[6, 5, 4, 3, 2, 1] >>> L.sort(); L # 方法:
就地排序[1, 2, 3, 4, 5, 6] >>> L.index(4) # 方法: 第一个
4 的偏移量 (搜索) 3 # 字典>>> {'a': 1,'b': 2}['b'] # 按键
索引字典2 >>> D = {'x': 1,'y': 2,'z': 3} >>> D['w'] = 0 #
创建新条目>>> D['x'] + D['w'] 1 >>> D[(1, 2, 3)] = 4 #
用元组 (不可变) 作键>>> D {'x': 1,'y': 2,'z': 3,'w': 0, (1,
2, 3): 4} >>> list(D.keys()), list(D.values()), (1, 2, 3) in
D # 键、值、键成员测试(['x','y','z','w', (1, 2, 3)], [1, 2, 3,
0, 4], True) # 空对象>>> [], ["", [], (), {}, None] # 各种
"空": 空对象们([[], ['', [], (), {}, None])

```

代码分析 (逐行解读)

```

>>> 2 ** 16                                # 65536
>>> 2 / 5, 2 / 5.0                         # / true division float
                                           #
>>> 'hack' + 'code'                       #
>>> S = 'Python'                          # S
>>> 'grok ' + S                            # 'grok Python'
>>> S * 5                                  # 5
>>> S[0], S[:0], S[1:]                    # 0 [0:0] 1
                                           # ('P', '', 'ython')
>>> 'coding %s is %s!' % (S, how)          # %
>>> 'coding {} is {}'.format(S, how)      # 2.6+ str.format

```

```

>>> f'coding {S} is {how}!'           # 3.6+ f-string
>>> ('x',)[0]                         # ('x',) 'x'
>>> L = [1, 2, 3] + [4, 5, 6]         # → [1,2,3,4,5,6]
>>> L[:0]                             # → []
>>> L[-2]                             # 2 → 5
>>> L[-2:]                           # → [5, 6]
>>> L.reverse(); L                   # None ; L
>>> L.sort(); L                      #
>>> L.index(4)                       # 4 3
>>> D['w'] = 0                       # ""=
>>> D[(1, 2, 3)] = 4                 #
>>> list(D.keys())                   # keys() view list()
>>> (1, 2, 3) in D                   # in → True
>>> [[], [], (), {}, None]          # ""None

```

深度理解

·核心概念：本练习覆盖三大容器家族——不可变序列（字符串、元组）、可变序列（列表）、映射（字典）——加上三种字符串格式化手段（%、format、f-string）。

·底层机制：reverse() 和 sort() 都是就地修改并返回 None——所以示例里用 ; 分隔、再单独求值 L 才能看到修改结果；这是 Python 设计者特意为之（防止你误以为返回新列表）。index() 是线性搜索。

·空切片返回同类型：S[:0] 是""、L[:0] 是 []——"空"永远和对象类型一致，这是很多算法的语法基础。

·字典键的约束：键必须可哈希（hashable），即不可变对象；元组可以、列表不行。in 对字典查的是键。

·常见误区：L.reverse() 不返回列表（返回 None）；D.keys() 不是列表而是视图；('x') 是字符串而 ('x',) 才是元组。

。

题 2. Indexing and slicing (索引与切片)

英文原文

2. Indexing and slicing: Indexing out of bounds (e.g., `L[4]`) raises an error; Python always checks to make sure that all offsets are within the bounds of a sequence.

On the other hand, slicing out of bounds (e.g., `L[-100:100]`) works because Python scales out-of-bounds slices so that they always fit (the limits are set to zero and the sequence length, if required).

Extracting a sequence in reverse, with the lower bound greater than the higher bound (e.g., `L[3:1]`), doesn't really work. You get back an empty slice (`[]`) because Python scales the slice limits to make sure that the lower bound is always less than or equal to the upper bound (e.g., `L[3:1]` is scaled to `L[3:3]`, the empty insertion point at offset 3).

Python slices are always extracted from left to right, even if you use negative indexes (they are first converted to positive indexes by adding the sequence length). Note that Python's three-limit slices modify this behavior somewhat. For instance, `L[3:1:-1]` does extract from right to left:

```
>> L = [1, 2, 3, 4] >> L[4] Traceback (most recent cal
```

```
l last): File "<stdin>", line 1, in <module> IndexError:
list index out of range >> L[-1000:100] [1, 2, 3, 4] >>
L[3:1] [] >> L [1, 2, 3, 4] >> L[3:1] = ['?'] >> L [1, 2, 3,
?', 4]
```

中文翻译

2. 索引与切片：越界索引（如 `L[4]`）会抛出错误；Python 总是检查所有偏移量是否落在序列边界之内。另一方面，越界切片（如 `L[-1000:100]`）是可行的，因为 Python 会把越界的切片“缩放”（scales）到总能适配的范围内（必要时把下限定为 0、上限定为序列长度）。要反向提取序列（下界大于上界，如 `L[3:1]`）其实行不通：你得到的是空切片（`[]`），因为 Python 会缩放切片边界，确保下界总是小于等于上界（例如 `L[3:1]` 被缩放成 `L[3:3]`，即偏移量 3 处的空插入点）。Python 切片总是从左到右提取，即使你使用负索引也不例外（负索引会先加序列长度转换成正索引）。注意 Python 的三参数切片会部分改变这一行为。例如 `L[3:1:-1]` 确实是自右向左提取：
`>>> L = [1, 2, 3, 4]`
`>>> L[4]` → `IndexError: list index out of range`（越界索引报错）
`>>> L[-1000:100]` → `[1, 2, 3, 4]`（越界切片被自动缩放）
`>>> L[3:1]` → `[]`（被缩放成 `L[3:3]`，空切片）
`>>> L[3:1] = ['?']` → 切片赋值：在位置 3 插入 '?'
`>>> L` → `[1, 2, 3, '?', 4]`

代码分析（逐行解读）

```

L = [1, 2, 3, 4]
L[4]           # 4 4 0..3 → IndexError
L[-1000:100]   # -1000 0100 4 →
L[3:1]         # 3 > 1 [3:3] → []
L[3:1] = ['?'] # = 3 '?'
L              # → [1, 2, 3, '?', 4]

```

深度理解

·核心概念：索引与切片是两种不同的操作——索引越界报错，切片越界自动缩放。这是 Python 精心设计的语义差异。

·底层机制：索引调用 `getitem(int)`，解释器先做边界检查；切片调用 `getitem(slice)`，内部先做“边界裁剪”（clamping）：负索引加序列长度，越界值裁剪到 `[0, len]` 区间。

·为什么这样设计：切片越界安全，让 `L[a:b]` 这类通用写法在递归、分治算法里无需防御性检查；而索引越界则往往意味着逻辑 bug，报错更利于调试。

·三参数切片的例外：`L[3:1:-1]` 的第三参数是步长（step），为负时从右向左提取，且下界/上界语义互换（start 大于 stop 时正常取）。

·切片赋值（slice assignment）：`L[3:1] = ['?']` 用空区间做插入；若区间非空则是替换或删除（见下一题）。

·常见误区：以为 `L[3:1]` 会反向返回 `[4, 3]`——它返回空列表；反向要用 `L[3:1:-1]`。

题 3. Indexing, slicing, and del (索引、切片与删除)

英文原文

3. Indexing, slicing, and del: Your interaction with the interpreter should look something like the following. Note that assigning an empty list to an offset stores an empty list object there, but assigning an empty list to a slice deletes the slice. Slice assignment expects another sequence, or you'll get a type error; it inserts items inside the sequence assigned, not the sequence itself:

```
>> L = [1, 2, 3, 4] >> L[2] = [] >> L [1, 2, [], 4] >> L[2:3] = [] >> L [1, 2, 4] >> del L[0] >> L [2, 4] >> del L[1:] >> L [2] >> L[1:2] = 1 Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: can only assign an iterable
```

中文翻译

3. 索引、切片与 del (删除) : 你与解释器的交互应该大致如下。注意：给偏移量赋空列表 ($L[2] = []$) 会在那里存一个空列表对象；但给切片赋空列表 ($L[2:3] = []$) 则是删除该切片。切片赋值期待另一个序列，否则你会得到类型错误；它把被赋序列的元素插入到序列内部，而不是插入序列本身：
 $\gg L = [1, 2, 3, 4] \gg L[2] = [] \rightarrow [1, 2, [], 4]$ (

索引赋值：把空列表对象存入偏移 2) >>> L[2:3] = [] → [1, 2, 4] (切片赋值空列表 = 删除该切片) >>> del L[0] → [2, 4] (删除偏移 0 的元素) >>> del L[1:] → [2] (删除从 1 到末尾的切片) >>> L[1:2] = 1 → TypeError: can only assign an iterable (切片赋值右侧必须是可迭代对象)

代码分析 (逐行解读)

```
L = [1, 2, 3, 4]
L[2] = []          # 2
                  # → [1, 2, [], 4]""
L[2:3] = []        # [2:3]
                  # → → [1, 2, 4]
del L[0]           # del → [2, 4]
del L[1:]          # del → [2]
L[1:2] = 1         # iterable
                  # 1 → TypeError
```

深度理解

·核心概念：列表的三种“改”操作语义截然不同——索引赋值（替换一个槽位）、切片赋值（用可迭代对象的元素展开替换一段，长度可变）、del（删除索引或切片，长度缩短）。

·底层机制：索引赋值走 `setitem(int, value)`；切片赋值走 `setitem(slice, iterable)`，解释器先展开可迭代对象再“挤进”区间，因此区间长度与右侧元素个数可以不同，导致列表增长或收缩。

·为什么 `L[2:3] = []` 会删除：把右侧空列表的 0 个元素“挤进”长度为 1 的区间，等价于删掉该区间。

·常见误区：分不清 `L[2] = []`（清空列表）与 `L[2:3] = []`（删元素）；以及忘记切片赋值右侧必须是可迭代对象（字符串、列表、range 都可以，整数不行）。

题 4. Tuple assignment (元组赋值)

英文原文

4. Tuple assignment: The values of X and Y are swapped. When tuples appear on the left and right of an assignment symbol (=), Python assigns objects on the right to targets on the left according to their positions. This is probably easiest to understand by noting that the targets on the left aren't a real tuple, even though they look like one; they are simply a set of independent assignment targets. The items on the right are a tuple, which gets unpacked during the assignment (this tuple provides the temporary assignment needed to achieve the swap effect): `>>> X='code'>>> Y='hack'>>> X, Y = Y, X>>> X'hack'>>> Y'code'`

中文翻译

4. 元组赋值：X 和 Y 的值被交换了。当元组出现在赋值符号 (=) 的左右两侧时，Python 会按位置把右侧的对象赋给左侧的目标。理解这一点最容易的方式是：注意到左侧的目标并不是真正的元组，尽管它们看起来像元组；它们只是一组相互独立的赋值目标。右侧的项才是一个元组，它会在赋值过程中被解包 (unpacked)（这个元组提供了实现交

换效果所需的临时赋值) : >>> X='code'>>> Y='hack'>>> X, Y = Y, X # 右侧先打包成元组 ('hack','code'), 再按位解包赋给 X、Y >>> X #'hack'>>> Y #'code'

代码分析 (逐行解读)

```
X = 'code'      # X  'code'
Y = 'hack'      # Y  'hack'
X, Y = Y, X      # 1) ('hack', 'code')
                  # 2) X = 'hack' Y = 'code'
X                # → 'hack'
Y                # → 'code'
```

深度理解

- 核心概念：元组解包 (unpacking) 赋值——右边先打包成临时元组，左边再按位置解包。它提供了"隐式临时变量"，使交换只需一行。
- 底层机制：X, Y = Y, X 编译成的字节码是先建立元组再索引解包 (如 UNPACKSEQUENCE 指令)，右侧求值先于左侧赋值完成，所以绝不出现"X 已被覆盖"的中间状态。
- 为什么左侧不是真元组：(X, Y) 写在赋值左侧时语法上就是"目标列表" (target list)，不是一个被创建的对象。
- 扩展：同一机制还支持 a, b, c = 序列 (解包任意序列)、a, rest = 序列 (星号收集多余项)、for k, v in dict.items() 等。
- 常见误区：初学担心"先赋 X 会丢旧值"——临时元组保证了原子性；另外元素个数不匹配会抛 ValueError。

题 5. Dictionary keys (字典键)

英文原文

5. Dictionary keys: Any immutable (technically, "hashable") object can be used as a dictionary key, including integers, tuples, strings, and so on. This really is a dictionary, even though some of its keys look like integer offsets. Mixed-type keys work fine, too: >>> D = {} >>> D[1] = 'a' >>> D[2] = 'b' >>> D[(1, 2, 3)] = 'c' >>> D {1:'a', 2:'b', (1, 2, 3):'c'}

中文翻译

5. 字典键：任何不可变（严格地说“可哈希” hashable）对象都可以用作字典键，包括整数、元组、字符串等等。这确实是一个字典，尽管其中有些键看起来像整数偏移量。混合类型的键也完全没问题：>>> D = {} >>> D[1] = 'a' # 整数键 >>> D[2] = 'b' # 整数键 >>> D[(1, 2, 3)] = 'c' # 元组键 >>> D {1:'a', 2:'b', (1, 2, 3):'c'}

代码分析（逐行解读）

```
D = {}                                #
D[1] = 'a'                            # 1""
D[2] = 'b'                            # 2
D[(1, 2, 3)] = 'c'                    #
D                                     # → {1: 'a', 2: 'b', (1, 2, 3): 'c'}
```

深度理解

·核心概念：字典键必须可哈希（hashable）——即对象

有稳定的 hash 值且可比较相等。整数、浮点数、字符串、元组（内部元素也全部可哈希）都可以；列表、字典、集合是可变对象，不可哈希、不能当键。

·底层机制：字典是哈希表（hash table）。查找键时先算 hash(key) 定位桶（bucket），再用 == 比较处理冲突。若键可变，哈希值会变化，导致条目“丢失”，所以语言层面禁止。

·为什么整数键合法：字典的键是“任意对象”，不是“从 0 开始的连续整数”；列表的整数索引只是它的特例语法。

·常见误区：以为键必须是字符串；以为元组只要包含一个列表也不能当键（确实不能——可哈希性要求元组内所有元素都可哈希）；用 in 测字典时查的是键而不是值。

题 6. Dictionary indexing（字典索引）

英文原文

6. Dictionary indexing: Indexing a nonexistent key (D['d']) raises an error; assigning to a nonexistent key (D['d']='hack') creates a new dictionary entry. On the other hand, out-of-bounds indexing for lists raises an error, too, but so do out-of-bounds assignments. Variable names work like dictionary keys; they must have already been assigned when referenced, but they are created when first assigned.

In fact, variable names can be processed as dictionary keys if you wish (they're made visible in the diction

aries of stack frames or module [or other object] namespaces):

```
>> D = {'a': 1, 'b': 2, 'c': 3} >> D['a'] 1 >> D['d'] Traceback  
(most recent call last): File "<stdin>", line 1, in <  
module> KeyError: 'd' >> D['d'] = 4 >> D {'a': 1, 'b': 2, '  
c': 3, 'd': 4} >> L = [0, 1] >> L[2] Traceback (most rece  
nt call last): File "<stdin>", line 1, in <module> Index  
Error: list index out of range >> L[2] = 3 Traceback (  
most recent call last): File "<stdin>", line 1, in <modu  
le> IndexError: list assignment index out of range
```

中文翻译

6. 字典索引：索引一个不存在的键（D['d']）会抛出错误；对不存在的键赋值（D['d']='hack'）则会创建新的字典条目。另一方面，列表的越界索引会报错，越界赋值同样报错。变量名的工作方式与字典键类似：它们在被引用之前必须已经被赋值，而在首次赋值时被创建。事实上，如果你愿意，变量名也可以当作字典键来处理（它们会出现在栈帧或模块（或其他对象）命名空间的字典里）：

```
>>> D = {'a': 1, 'b': 2, 'c': 3} >>> D['a'] → 1（读存在的键）>>> D['d'] →  
KeyError: 'd'（读不存在的键：报错）>>> D['d'] = 4 →  
对不存在的键赋值：创建新条目>>> D → {'a': 1, 'b': 2, 'c':  
3, 'd': 4} >>> L = [0, 1] >>> L[2] → IndexError: list ind  
ex out of range（列表越界索引报错）>>> L[2] = 3 → I  
ndexError: list assignment index out of range（列表越  
界赋值也报错）
```

代码分析（逐行解读）

```
D['a']          # → 1
D['d']          # → KeyError'd'
D['d'] = 4      # → ""
L[2]           # → IndexError
L[2] = 3       # → IndexError"" append
```

深度理解

- 核心概念：读与写的语义不同——字典读不存在键报 `KeyError`，写不存在键则“惰性创建”；列表读写越界都报 `IndexError`。原因：字典是映射（键集合自由扩张），列表是定长序列（下标必须落在 $0..len-1$ ）。
- 变量名即字典键：Python 的实现里，命名空间本身就是字典——模块的 `dict`、函数栈帧的 `flocals` 都是字典。“变量未定义”在底层就是“字典里没有这个键”。所以 `dir()`、`globals()` 能看到所有变量名。
- 常见误区：把字典当成“序号数组”用；或以为列表能像字典一样自动扩容——列表必须显式 `append/extend`。

题 7. Generic operations（通用操作）

英文原文

7. Generic operations: Question answers (with some error text omitted in listings): a. The `+` operator doesn't work on different/mixed types (e.g., `string + list`, `list + tuple`).

b. + doesn't work for dictionaries, as they aren't sequences (though | does).

c. The append method works only for lists, not strings, and keys works only on dictionaries. append assumes its target is mutable, since it's an in-place extension; strings are immutable. Dictionary keys is similarly type specific.

d. Slicing and concatenation always return a new object of the same type as the objects processed:

```
>>'x' + 1 TypeError: illegal argument type for built-in operation
>> {} + {} TypeError: bad operand type(s) for +
>> [].append(9) >> ''.append('s') AttributeError: attribute-less object
>> list({}.keys()) [] >> [].keys() AttributeError: keys
>> [][:] [] >> "[:]"
```

中文翻译

7. 通用操作：问答答案如下（列表中的部分错误文本已省略）：a. + 运算符不能用于不同类型/混合类型（例如 string + list、list + tuple）。b. + 对字典不适用，因为字典不是序列（不过 | 可以）。c. append 方法只适用于列表，不适用于字符串；keys 只适用于字典。append 假定其目标是可变的，因为它是就地扩展；而字符串是不可变的。字典的 keys 同样具有类型特异性。d. 切片与拼接总是返回一个与所处理对象同类型的新对象：>>>'x' + 1 → TypeError（混合类型相加：非法）>>> {} + {} → TypeError（字典不是序列，不支持 +）>>> [].append(9) → 就地追加，

返回 None >>> ".append('s') → AttributeError (字符串没有 append 方法) >>> list({}.keys()) → [] (空字典的键转列表) >>> [].keys() → AttributeError (列表没有 keys 方法) >>> [][:] → [] (空切片返回同类型空列表) >>> "[:] → " (空切片返回同类型空字符串)

代码分析 (逐行解读)

```
'x' + 1          # str int __add__ → TypeEr...
{} + {}          # dict __add__ ""→ TypeError
[].append(9)     # 9 None None
''.append('s')   # append → AttributeError
list({}.keys())  # keys() → list() []
[].keys()        # keys → AttributeError
[][: ]          # → []
''[: ]          # → ''
```

深度理解

·核心概念：Python 是强类型语言——运算符和方法大多类型特化，不同类型之间拒绝隐式转换或混用。切片与拼接是"保型"操作（返回同类型新对象）。

·底层机制：+ 走二元运算符协议 (add / radd)，每个类型自己决定支持与否；方法 (append、keys) 只存在于对应类型的方法表里。切片走 getitem(slice)，类型自己构造同类型返回值。

·为什么字典没有 +: + 的语义是"序列拼接"（保持顺序）；字典是无序映射，没有拼接概念。Python 3.9+ 给字典补了 | (合并) 运算符作为替代。

·字符串为什么没有 append：字符串不可变；就地扩展会破坏不可变性，所以设计者根本不提供该方法（想"改"字

字符串要用拼接或 bytearray) 。

·常见误区：以为 "abc" + 1 会自动把 1 转成字符串（不会，必须显式 str(1)）；以为所有容器都有 keys/append。

题 8. String indexing (字符串索引)

英文原文

8. String indexing: This is a bit of a trick question—because strings are collections of one-character strings, every time you index a string, you get back a string that can be indexed again. `S[0][0][0][0][0]` just keeps indexing the first character over and over. This generally doesn't work for lists (lists can hold arbitrary objects) unless the list contains strings: `>>> S='hack'>>> S[0][0][0][0][0]'h'>>> L=['h','a']>>> L[0][0][0]'h'`

中文翻译

8. 字符串索引：这算一道脑筋急转弯——因为字符串是“单字符字符串”的集合，每次索引字符串得到的都是可再被索引的字符串。`S[0][0][0][0][0]` 只是一遍又一遍地索引第一个字符。这对列表通常不成立（列表可以装任意对象），除非列表里恰好是字符串：`>>> S='hack'>>> S[0][0][0][0][0]→'h'（每次索引都取第 0 个字符，结果还是'h'）>>> L=['h','a']>>> L[0][0][0]→'h'（L[0] 是字符串'h'，因此还能继续索引）`

代码分析（逐行解读）

```
S = 'hack'
S[0][0][0][0][0]    # S[0] → 'h''h'[0] → 'h'..... 'h'
L = ['h', 'a']
L[0][0][0]           # L[0] → 'h''h'[0] → 'h' 'h'
```

深度理解

- 核心概念：字符串的"原子"也是字符串（单字符字符串）——类型系统上的"递归自包含"，让索引链永不失效。
- 底层机制：'h'[0] 返回一个新的单字符字符串对象（CPython 会复用内部缓存的小字符串），其本身仍是字符串，支持 `getitem`，所以可以无限索引下去。
- 列表的区别：L[0] 返回的是任意对象；若 L[0] 是整数，再索引 L[0][0] 就报 `TypeError`（int 不可索引）。
- 常见误区：误以为字符串能"索引到字符的字符"——字符串没有更底层的单元，字符就是字符串本身。

题 9. Immutable types（不可变类型）

英文原文

9. Immutable types: Either of the following solutions works. Index assignment doesn't because strings are immutable: `>>> S='hack'>>> S = S[0] + 'e' + S[2:] >>> S'heck'`
`>>> S = S[0] + 'i' + S[2] + S[3] >>> S'hick'`
(See also the bytearray string type in Chapter 37—it's a mutable sequence of small integers that is essential

ly processed the same as a string, especially when its bytes are ASCII character code points.)

中文翻译

9. 不可变类型：下面两种方案都可以。索引赋值不行，因为字符串是不可变的：>>> S = 'hack'>>> S = S[0] + 'e' + S[2:] # 拼接法：'h' + 'e' + 'ck' → 'heck'>>> S'heck'>>> S = S[0] + 'i' + S[2] + S[3] # 逐字索引拼接：'h' + 'i' + 'c' + 'k' → 'hick'>>> S'hick'（另见第 37 章的 bytearray 字符串类型——它是小整数的可变序列，处理方式本质上与字符串相同，尤其当它的字节是 ASCII 字符码点时。）

代码分析（逐行解读）

```
S = 'hack'
S = S[0] + 'e' + S[2:]          # S
                                # S[0]='h'S[2:]='ck' → 'h' + 'e' + ...
S = S[0] + 'i' + S[2] + S[3]   # 'h'+'i'+'c'+'k' = 'hick'
```

深度理解

- 核心概念：字符串不可变 (immutable) ——没有"原地修改"操作。任何"修改"都是构造新字符串再重新赋值。
- 底层机制：CPython 中字符串是 PyUnicodeObject，内部没有 setitem 修改路径；S[0] = 'e' 会直接抛 TypeError: 'str' object does not support item assignment.
- 为什么这样设计：不可变性让字符串可以安全共享（驻留）、哈希、用作字典键、跨线程传递。
- bytearray 的补充：bytearray 是可变的字节序列，需要频繁修改字符串内容时可先转 bytearray 改完再转回。

·常见误区：试图 `S[0] = 'e'`；或在循环里用 `s += c` 大量拼接（产生 $O(n^2)$ 临时对象），应改用 `".join()"`。

题 10. Nesting（嵌套）

英文原文

```
10. Nesting: Here is a sample (your specs will vary): >
>> pat = {'name': ('Pat','Q','Jones'),'age': None,'job':'engineer'}
>>> pat['job']
'engineer'
>>> pat['name'][2]
'Jones'
```

中文翻译

10. 嵌套：这里是一个样例（你的规格可能会不同）：

```
>>
> pat = {'name': ('Pat','Q','Jones'),'age': None,'job':'engineer'}
>>> pat['job'] → 'engineer'（按键取值）
>>> pat['name'][2] → 'Jones'（值本身是元组，再按位置取第 2 项）
```

代码分析（逐行解读）

```
pat = {'name': ('Pat', 'Q', 'Jones'),    # ""
       'age': None,                      # None
       'job': 'engineer'}
pat['job']                                # → 'engineer'
pat['name'][2]                            # pat['name'] [2] 'Jones'
```

深度理解

·核心概念：容器可以任意嵌套——字典里装元组、元组里装列表、列表里装字典……“对象组合成对象”正是 Python

数据的精髓。

·访问链: `pat['name'][2]` 是链式索引, 每一步返回的对象决定下一步的索引语义——这是"表达式从左到右逐步求值"的体现。

·实际价值: 用嵌套结构可以无类地表达记录 (record) 数据; 需要更严格的结构时可升级为类 (Part VI) 。

·常见误区: `age: None` 表示"未赋值", 不是 0 也不是空字符串; 嵌套的共享引用问题 (改内部对象会影响所有引用它的容器) 。

题 11. Files (文件)

英文原文

11. Files: Examples B-3 and B-4 show one way to create and read back a text file in Python using Unicode encoding defaults on the host (which are generally most for simple ASCII text like this): Example B-3. Part2/maker.py

```
python file = open('myfile.txt','w') file.write('Hello file world!\n') # Or: open().write() file.close() # close not always needed
```

Example B-4. Part2/reader.py

```
python file = open('myfile.txt') #'r' is default open mode print(file.read()) # Or print(open().read()) When run (here, from a console command line), the file sho
```

ws up in the directory you're working in because its name has no path prefix. The ls here is a Unix command; use dir on Windows:

```
$ python3 maker.py $ python3 reader.py Hello file world!
```

```
$ ls -l myfile.txt-rw-r--r-- 1 me staff 18 Aug 11 19:34 myfile.txt
```

中文翻译

11. 文件：示例 B-3 和 B-4 展示了在 Python 中创建并读回文本文件的一种方式，使用主机环境的 Unicode 编码默认值（对这样的简单 ASCII 文本来说，编码默认值一般无关紧要）：示例 B-3. Part2/maker.py（写文件）
`python file = open('myfile.txt','w') file.write('Hello file world!\n') # 或者：open().write() file.close() # close 并不总需要`
示例 B-4. Part2/reader.py（读文件）
`python file = open('myfile.txt') # 'r' 是默认打开模式 print(file.read()) # 或者 print(open().read())`
运行时（这里从控制台命令行），文件会出现在你工作的目录中，因为文件名没有路径前缀。这里的 ls 是 Unix 命令；Windows 上用 dir：
`$ python3 maker.py $ python3 reader.py Hello file world! $ ls -l myfile.txt-rw-r--r-- 1 me staff 18 Aug 11 19:34 myfile.txt`

代码分析（逐行解读）

```
# maker.py —
file = open('myfile.txt', 'w') # 'w' =
file.write('Hello file world!\n') # '\n'
file.close() #
# close

# reader.py —
file = open('myfile.txt') # 'r'
print(file.read()) # read() print
```

深度理解

- 核心概念：文件操作三步骤——open（打开）→ 读/写 → close（关闭）。open 的第一个参数是路径，第二个是模式（'r' 读、'w' 写、'a' 追加、'rb'/'wb' 二进制）。
- 底层机制：文件对象是缓冲 IO 流：写入先进内存缓冲，close()（或 flush()）时才真正落盘；read() 返回整个文件字符串。Python 3 文本模式默认按平台编码（UTF-8）编解码 Unicode。
- 为什么 close "不总需要"：CPython 引用计数归零时对象会自动关闭文件；但显式关闭（或用 with open(...) as f:）是良好习惯——可移植性更好、可预测。
- 相对路径：文件名无路径前缀 → 相对当前工作目录（运行命令所在目录）。ls/dir 可确认文件生成。
- 常见误区：以'w' 打开会清空原文件内容；忘记写'\n' 导致多行内容挤一行；print(open(...)) 打印的是文件对象而不是内容（必须 .read()）。

Part III: Statements and Syntax (语句与语法)

英文原文

Part III, Statements and Syntax See "Test Your Knowledge: Part III Exercises" in Chapter 15 for the exercises.

中文翻译

第三部分：语句与语法。练习见第 15 章的 "Test Your Knowledge: Part III Exercises"。

深度理解

·核心概念：Part III 练习从"对象层"上升到"语句层"——for/while 循环、if 选择、match 语句、字典排序、字符串转义字符，以及同一逻辑的多种等价写法 (if/match/dict/list) 。

题 1. Coding basic loops (编写基础循环)

英文原文

1. Coding basic loops: As you work through this exercise, you'll wind up with code that looks like the following:

```
>> S = 'hack'>> for c in S:
```

```

... print(ord(c)) ... 104 97 99 107 >> x = 0 >> for c in
S: x += ord(c) # Or: x = x + ord(c)

... >> x 407 >> chr(x) # Extra credit: non-ASCII, see C
hapter 37">> x = [] >> for c in S: x.append(ord(c))
# Manual list construction

... >> x [104, 97, 99, 107] >> list(map(ord, S)) [115, 11
2, 97, 109] >> [ord(c) for c in S] # map and listcomps
automate list builders [115, 112, 97, 109]

```

中文翻译

1. 编写基础循环（循环的走向）：等你做完全部步骤，你会得到类似下面的代码：

```

>>> for c in S: print(ord(c)) #
逐个字符打印其 Unicode 码点 (ord) 104 #'h'97 #'a'99
#'c'107 #'k'>>> x = 0 >>> for c in S: x += ord(c) # 或
: x = x + ord(c)——累加所有码点407 >>> chr(x) # 加
分项：非 ASCII 码点，见第 37 章">>> x = [] >>> for
c in S: x.append(ord(c)) # 手动构造列表[104, 97, 99, 10
7] >>> list(map(ord, S)) # map 内置调用：自动化构造
列表[115, 112, 97, 109] >>> [ord(c) for c in S] # 列表
推导式：同样自动化[115, 112, 97, 109]

```

（注意：原文后两条打印的是 map/列表推导式作用于'spam' 之类字符串的示例输出，若你对'hack' 求值，结果会是 [104, 97, 99, 107]；本质都是"取每个字符的码点"。）

代码分析（逐行解读）

```
S = 'hack'
for c in S:                                #
    print(ord(c))                          # ord() → Unicode

x = 0
for c in S:
    x += ord(c)                            # x = x + ord(c)
x                                           # → 407
chr(x)                                     # chr() → 407  ASCII  'I'

x = []
for c in S:
    x.append(ord(c))                       #
x                                           # → [104, 97, 99, 107]

list(map(ord, S))                          # map(, ) ord  S
                                           # Python 3  map  list() ...

[ord(c) for c in S]                        # ""
```

深度理解

·核心概念：同一个任务（把每个字符转成码点）有三种表达方式——手写 for+append、map 内置、列表推导式。三者结果等价，但风格与性能不同。

·底层机制：for c in S 走迭代协议（S.iter）；ord 是 C 实现的内置函数；map(ord, S) 逐元素应用函数并惰性产出；列表推导 [expr for c in S] 编译成专门的快速循环。

·为什么把 range/map 的「结果可能一样」当作练习点：这是 Python 由“过程式”通往“函数式/声明式”的桥梁概念，为 Part IV 的推导式、生成器做铺垫。

·常见误区：Python 3 里 map 返回的是迭代器而非列表，直接打印只会看到 <map object at ...>；忘记在 for 循环后换行缩进。

题 2. Coding basic selections (编写基础选择)

英文原文

2. Coding basic selections: Here is the sort of code expected. To handle out-of-range numbers, add an else for if, a case for match, a get method call or in test for the dictionary, and a try handler for the list. For versions of this code that are easier to copy/paste, see file Part3/selections.txt in the examples package:

```
>>> month = 3 >>> if month == 1: ... print('January') ... elif month == 2: ... print('February') ... elif month == 3: ... print('March') ... March
```

```
>>> match month: ... case 1: ... print('January') ... case 2: ... print('February') ... case 3: ... print('March') ... March
```

```
>>> {1:'January', 2:'February', 3:'March'}[month]'March'
```

```
>>> ['January','February','March'][month - 1]'March'
```

中文翻译

2. 编写基础选择：下面是期望的代码形态。为了处理越界数字：if 加个 else, match 加个 case, 字典用 get 方法或 in 测试, 列表用 try 处理。示例包里 Part3/selections.txt 有更容易复制/粘贴的版本：

```
>>> month = 3
>>> if month == 1: # elif 链：从上到下匹配... print('January') ...
elif month == 2: ... print('February') ...
elif month == 3: ... print('March')
March
>>> match month: # Python 3.10+ 的结构化模式匹配...
case 1: ... print('January') ...
case 2: ... print('February') ...
case 3: ... print('March')
March
>>> {1:'January', 2:'February', 3:'March'}[month]
# 字典查表'March'
>>> ['January', 'February', 'March'][month - 1]
# 列表索引'March'
```

代码分析 (逐行解读)

```
month = 3

if month == 1: print('January')    # if/elif
elif month == 2: print('February')
elif month == 3: print('March')    # → March

match month:                        # match if
    case 1: print('January')
    case 2: print('February')
    case 3: print('March')

{1: 'January', 2: 'February', 3: 'March'}[month]    # → 'Mar...'
['January', 'February', 'March'][month - 1]         # 0 mo...
```

深度理解

·核心概念：同一个“分类”问题有四种表达：if/elif 链（过

程式)、match/case (结构化模式匹配, 3.10+)、字典查表 (映射式)、列表索引 (序号式)。

·边界处理方式的差异: if 需 else 兜底; match 需 case (通配) 兜底; 字典越界读报 KeyError, 可用 dict.get(key, 默认) 或先 in 判断; 列表越界报 IndexError, 可用 try/except 捕获或用 month-1 算好再检查范围。

·设计思想: 字典查表是"数据驱动编程"的典范——把分支逻辑转化为数据映射, 可读性、扩展性都好。

·常见误区: 列表索引漏掉 -1 (month=1 会取到'January' 才对, 直接 [month] 会错位/越界); match 是表达式作用于值, 不要写成 == 链式的误用。

题 3. Backslash characters (反斜杠字符)

英文原文

3. Backslash characters: The example prints the bell character (\a) 50 times. Assuming your machine can handle it, and when it's run outside of some interfaces like IDLE, you may get a series of beeps (or one sustained tone if your machine is fast enough). Hey—you were warned.

中文翻译

3. 反斜杠字符: 示例打印 50 次响铃字符 (\a)。假设你的机器能处理它, 并且它在 IDLE 等一些界面之外运行, 你可能会听到一串"哔"声 (如果你的机器足够快, 可能是一个持

续的长音)。嘿——这可是警告过你的哦。

代码分析

```
print('\a' * 50)    # 50 ASCII 7, BEL
```

深度理解

- 核心概念：反斜杠 (\) 在字符串字面量里是转义字符前缀：\a 是响铃、\n 是换行、\t 是制表符、\\ 是字面反斜杠、\" 是转义引号。
- 底层机制：转义在编译期完成——'\a' 在字节码里就是一个含单个 BEL 字符的字符串；50 字符串重复产生 50 个 BEL。
- 实际输出：终端碰到 BEL 字符会发哔声/闪屏；IDLE 等图形界面的文本控件往往忽略它。
- 常见误区：Windows 路径'C:\new' 里的 \n 会被当成换行符——用 raw 字符串 r'C:\new' 或双反斜杠。

题 4. Sorting dictionaries (字典排序)

英文原文

4. Sorting dictionaries: Here's one way to work through this exercise (see Chapter 8 or Chapter 14 if this doesn't make sense). You really do have to split off the keys and sort calls like this because sort returns None. You can iterate through dictionary keys directly without calling keys (e.g., for key in D:), but the keys list

```
will not be sorted like it is by this code. The sorted built-in is simpler but creates a new list object: >>> D = {'a': 1, 'c': 3, 'e': 5, 'g': 7, 'f': 6, 'd': 4, 'b': 2} >>> D {'a': 1, 'c': 3, 'e': 5, 'g': 7, 'f': 6, 'd': 4, 'b': 2} >>> keys = list(D.keys()) # Keys view has no sort method >>> keys.sort() # Sort list in place: returns None >>> for key in keys: ... print(key, '=>', D[key]) ... a => 1 b => 2 c => 3 d => 4 e => 5 f => 6 g => 7 >>> D {'a': 1, 'c': 3, 'e': 5, 'g': 7, 'f': 6, 'd': 4, 'b': 2} >>> >>> for key in sorted(D): # Simpler alternative, but a new list ... print(key, '=>', D[key]) ... ...same output...
```

中文翻译

4. 字典排序：解决这个练习的一种方法如下（看不懂就回头翻第 8 章或第 14 章）。你确实必须像这样把 `keys` 和 `sort` 这两个调用拆开：因为 `sort` 返回的是 `None`。你可以直接遍历字典（例如 `for key in D:`），不需要调用 `keys`，但那样遍历的键不会像这段代码那样有序。内置的 `sorted` 更简单，但会创建新的列表对象：>>> D = {...} # 注意：字典本体顺序不变（内部实际为键插入序/哈希序）>>> keys = list(D.keys()) # dictkeys 视图没有 `sort` 方法，必须先转成 `list`>>> keys.sort() # 列表就地排序，返回 `None`>>> for key in keys: print(key, '=>', D[key]) a => 1 b => 2...g => 7 # 按键字母序输出>>> for key in sorted(D): # 更简单的替代：对可迭代对象排序并返回新列表... print(key, '=>', D[key])...同样输出...

代码分析（逐行解读）

```
D = {'a': 1, 'c': 3, 'e': 5, 'g': 7, 'f': 6, 'd': 4, 'b': 2}
keys = list(D.keys()) # 1) D.keys() dict_keys sort
                        # 2) list(...)
keys.sort()           # 3) None
for key in keys:      # 4)
    print(key, '=>', D[key])
# a=>1 b=>2 c=>3 d=>4 e=>5 f=>6 g=>7

for key in sorted(D): # sorted →
    print(key, '=>', D[key])
#
```

深度理解

- 核心概念：字典本身无序（历史上是哈希序；Python 3.7 + 保插入序但不"排序"），想按键排序必须借助"键列表 + 排序"。
- 为什么不能 `D.keys().sort()`：dictkeys 视图只提供只读迭代，没有 `sort` 方法；必须先 `list()` 复制成真正的列表。
- 为什么 `keys.sort()` 要单独一行：`list.sort()` 是就地排序、返回 `None`，所以 `print(keys.sort())` 会打印 `None`——这是 Python 最具名的"陷阱"之一。
- 更优做法：`sorted(D)` 一步到位（内置函数返回新列表）；`for key in sorted(D)` 是惯用法。
- 常见误区：以为字典"本来就是有序的"（只是 3.7+ 保证插入序，不等于排序）；把就地方法当返回新对象用。

题 5. Program logic alternatives (程序逻辑的多种等价写法)

英文原文

5. Program logic alternatives: Here's some sample code for the solutions, available in the examples package's Part3/power.py. For step e, assign the result of $2X$ to a variable outside the loops of steps a and b and use it inside the loop. Your results may vary; this exercise is mostly designed to get you playing with code alternatives, so anything reasonable gets full credit:

```
python # a L = [1, 2, 4, 8, 16, 32, 64] X = 5 i = 0 while i < len(L): if 2 X == L[i]: print('at index', i) break i += 1 else: print(X,'not found') # b L = [1, 2, 4, 8, 16, 32, 64] X = 5 for p in L: if (2 X) == p: print((2 X),'was found at ', L.index(p)) break else: print(X,'not found')
```

中文翻译

5. 程序逻辑的多种写法：下面是这些解答的样例代码，位于示例包的 Part3/power.py。对于步骤 e ("深入思考")，把 $2X$ 的值赋给循环外的变量，再在循环里使用它。你的结果可能会不同；这个练习的初衷主要是让你玩一玩代码的各种等价写法，所以任何合理的解法都算满分：

```
# a: while + 手动索引 + break + else L = [1, 2, 4, 8, 16, 32, 64] X = 5 i = 0 while i < len(L): if 2 X == L[i]: print('at index', i) break i += 1 else: print(X,'not found') # b: for + L.index 查位置 L = [1, 2, 4, 8, 16, 32, 64] X = 5 for p
```

```
in L: if (2 X) == p: print((2 X), 'was found at', L.index(p))
) break else: print(X, 'not found')
```

题 5 (续) : c、d、e 与完整代码

这部分和后面的选项在答案原料里一起给出的更完整代码整理如下（含 if X in L 的第三种写法、构造列表的第四种、以及“深入思考”（map/列表推导式）的第五种）。它们的目的是演示用不同方式表达“在一个列表里找 2 的 X 次方”：

```
# c
L = [1, 2, 4, 8, 16, 32, 64]
X = 5
if (2 ** X) in L:                                # in
    print((2 ** X), 'was found at', L.index(2 ** X))
else:
    print(X, 'not found')

# d
X = 5
L = []
for i in range(7):
    L.append(2 ** i)                             # 2**0..2**6
print(L)                                         # [1, 2, 4, 8, 16, 32, 64]
if (2 ** X) in L:
    print((2 ** X), 'was found at', L.index(2 ** X))
else:
    print(X, 'not found')

# "Deeper thoughts" (e)
X = 5
L = list(map(lambda x: 2 ** x, range(7)))      # [2 ** x for x in r...
print(L)
if (2 ** X) in L:
```

```
print((2 ** X), 'was found at', L.index(2 ** X))
else:
    print(X, 'not found')
```

深度理解

· 写法对比：a 用 while + break + 循环 else（不被 break 才打印 not found）；b 用 for 直接遍历值并用 L.index(p) 反查位置；c 用 in 一次性判断加查位置；d 先动态构造 2 的幂列表再查；e 用 map(lambda...) 或列表推导式一行构造列表。

· while 的 else 子句：Python 特有语法——循环没有被 break 提前终止时才执行 else，非常优雅地处理了“找不到”分支。

· 设计思想：同一逻辑五种表达，锻炼“用多种视角看问题”；e 的“把结果变量提出来在循环外算一次”是性能技巧（避免循环里重复计算 2X）。

· 常见误区：L.index(p) 是线性搜索，代价高；in 也是线性。a、b 的 else 语义与 for/while 绑定，与 if 的 else 无关。

Part IV: Functions and Generators (函数与生成器)

英文原文

Part IV, Functions and Generators See "Test Your Knowledge: Part IV Exercises" in Chapter 21 for the exerci

中文翻译

第四部分：函数与生成器。练习见第 21 章的 "Test Your Knowledge: Part IV Exercises"。

深度理解

·核心概念：Part IV 练习覆盖函数的多态、参数传递（位置/关键字/args/kwargs/默认值）、参数匹配规则、递归、推导式与 map、计时工具（timer2.py）、素数、阶乘，以及"标准库 vs 手写 vs 递归"的效率权衡。

题 1. The basics (基础)

英文原文

1. The basics: There's not much to this one, but notice that using print (and hence your function) is technically a polymorphic operation, which does the right thing for each type of object:

```
$ python3 >>> def echo(x): ... print(x) ... >>> echo('hack') hack
```

```
>>> echo(3.12) 3.12
```

```
>>> echo([1, 2, 3]) [1, 2, 3]
```

```
>>> echo({'edition': 6}) {'edition': 6}
```

中文翻译

python >>> def echo(x): # 函数体只有一句 print(x): x 的类型不被声明... print(x) >>> echo('hack') → hack (字符串) >>> echo(3.12) → 3.12 (浮点数) >>> echo([1, 2, 3]) → [1, 2, 3] (列表) >>> echo({'edition': 6}) → {'edition': 6} (字典, 注意这里只是键=6) 这一点就是多态 (polymorphism) : print (以及你的函数) 对每种对象都"做正确的事"。

(题目正文只有这两段：答案要点在于多态。)

代码分析 (逐行解读)

```
def echo(x):          # x —
    print(x)          # print __str__

echo('hack')          # str __str__ → hack
echo(3.12)             # float __str__ → 3.12
echo([1, 2, 3])        # list __str__ → '[1, 2, 3]'
echo({'edition': 6})   # dict __str__ → "{'edition': 6}"
```

深度理解

·核心概念：多态性 (polymorphism) ——同一个接口 (函数) 作用于不同类型的对象，行为各自正确。"鸭子类型 (duck typing)": 只要对象支持所需的操作 (这里是 str)，就能被 print 用。

·底层机制：print 内部调用 str(x)，即 x.str()——每种内置类型都实现了 str；用户自定义类也可覆写它定制打印格式。所以"能做对的事"不是魔法，而是协议 (protocol) 约定。

- 为什么这样设计：Python 弱化类型标注，让你无需为每种类型重写一遍打印逻辑；适合快速开发与通用工具库。
- 常见误区：多态 $\neq$ 隐式类型转换。echo(1) + echo('x') 仍会报类型错误，"多态"是针对"都能被打印"，不是"到处都是同一个类型"。

题 2. Arguments (参数与多态加法)

英文原文

```
python # Part4/adder1.py def adder(x, y): return x + y
print(adder(5, 1.0)) # 6.0 print(adder('hack','code'))
# hackcode print(adder(['a','b'], ['c','d'])) # ['a','b','c','d']
'] And the output:
```

```
$ python3 adder1.py 6.0 hackcode ['a','b','c','d']
```

中文翻译

示例 B-5 给出一个样例方案。记住：必须用 print 才能在测试调用里看到结果，因为文件不同于交互式输入的代码；Python 不会像在 >>> 提示符下那样回显文件里表达式语句的结果：python def adder(x, y): return x + y print(adder(5, 1.0)) # 6.0（数值相加）print(adder('hack','code')) # hackcode（字符串拼接）print(adder(['a','b'], ['c','d'])) # ['a','b','c','d']（列表拼接）

深度理解

- 核心概念：return $x + y$ 里 $+$ 也是多态的——对数值做算术、对字符串/列表做拼接。同一个函数签名服务多种类型。
- 为什么强调文件必须 print：交互模式（REPL）会自动回显表达式结果；文件执行时只执行语句，表达式结果被丢弃。这是新手最大的困惑之一。
- 底层：int.add、str.add、list.add 都是各类型自己实现的"加法"，而函数本身不关心——这是"协议传调"的设计。
- 常见误区：把 'hack' + 1 误想成自动转类型（不会，会 TypeError）；忘了在文件里 print 结果。

题 3. Arbitrary arguments（任意个数的位置参数）

练习：一个可以接收任意多参数、并返回它们之和的 adder 函数版本。

分析纪要

- 难点：如何把累加器初始化为"传入类型对应的空值"。
- 解法 1（adder1）：判断第一个参数是否为整数（用 `type(args[0]) == type(0)`），是则初始化为 0；否则假定为序列，用空切片 `args[0][:0]` 得到同类空值（''、[]）。然

后 for 逐个 +。

·解法 2 (adder2)：更优——直接用第一个参数初始化，从 args[1:] 开始累加（类似第 18 章 min 的变体）；无需显式类型判断。

·两者都假定所有实参同类型；都不适用于字典（字典无 +，会报 TypeError）；都有更简单的 sum(iterable) 内置方案——但练习的初衷是让你亲手写出自己的求和函数。

·陷阱：args[0][:0] 需要假定的传入的是“可切片”的序列；若是 dict 将直接 TypeError。

代码

```
# adder2.py B-6
def adder1(*args):
    print('adder1:', end=' ')
    if type(args[0]) == type(0):    # int
        sum = 0                    # 0
    else:                           #
        sum = args[0][:0]          # ""
    for arg in args:
        sum = sum + arg
    return sum

def adder2(*args):
    print('adder2: ', end=' ')
    sum = args[0]                  #
    for next in args[1:]:         #
        sum += next               #
    return sum

for func in (adder1, adder2):
    print(func(2, 3, 4))
    print(func('hack', 'code', 'well'))
```

```
print(func(['a', 'b'], ['c', 'd'], ['e', 'f']))
```

```
$ python3 adder2.py
adder1: 9
adder1: hack codewell
adder1: ['a', 'b', 'c', 'd', 'e', 'f']
adder2: 9
adder2: hack codewell
adder2: ['a', 'b', 'c', 'd', 'e', 'f']
```

深度理解

- args 收集: args 把任意数量的位置实参打包为一个元组。两种初始化的思路值得学习。
- 解法 2 更优的理由: 免去类型判断, 靠四种类型各自的 + 语义 (多态) 自然得到正确结果——代码更短、更通用。
- 局限与延伸: 假定类型一致; 想支持字典合并需改写 (如 for/update//|) ; sum() 内置只支持数值/可加; 学习的重点在于"自己造轮子"。
- 运行文件的原因: 文件里没有自动回显, 所以函数里加了 print 前缀才输出标记。

题 4. Keywords (关键字与默认值)

练习原文简译 (含示例 B-7 / B-8)

- 定义 adder3.py: 默认值三个参数 red=1, green=2, blue=3, 返回三者之和, 测试各种调用方式。
- 定义 adder4.py: 四种版本求和 (args 位置收集版、arg

s 关键字收集版 ×3，分别用 .keys() 然后逐个访问、用 .values() 转换、或直接复用 adder1(args.values()))，验证结果一致。

代码（示例 B-7 / B-8）

```
# adder3.py —
def adder(red=1, green=2, blue=3):
    return red + green + blue

print(adder())           # 1+2+3 = 6
print(adder(5))          # red=5+2+3 = 10
print(adder(5, 6))       # 5+6+3 = 14
print(adder(5, 6, 7))    # 5+6+7 = 18
print(adder(blue=7, red=6, green=5)) # 6+5+7 = 18
print(adder(blue=1, red=2)) # 2+2+1 = 5
```

```
$ python3 adder3.py
6
10
14
18
18
5
```

```
# adder4.py — *args **args
def adder1(*args):           #
    tot = args[0]
    for arg in args[1:]:
        tot += arg
    return tot

def adder2(**args):          #
    argskeys = list(args.keys()) # list()
    tot = args[argskeys[0]]
    for key in argskeys[1:]:
        tot += args[key]
```

```

        return tot

def adder3(**args):                #
    args = list(args.values())      # list()
    tot = args[0]
    for arg in args[1:]:
        tot += arg
    return tot

def adder4(**args):                #
    return adder1(*args.values())

print(adder1(1,2,3), adder1('aa','bb','cc'))
print(adder2(a=1,b=2,c=3), adder2(a='aa',b='bb',c='cc'))
print(adder3(a=1,b=2,c=3), adder3(a='aa',b='bb',c='cc'))
print(adder4(a=1,b=2,c=3), adder4(a='aa',b='bb',c='cc'))

```

```

$ python3 adder4.py
6 aabbcc
6 aabbcc
6 aabbcc
6 aabbcc

```

深度理解

- 默认值：函数定义时的缺省实参值；调用时按位/按名覆盖。注意默认值在定义时求值一次（后面可变默认值是陷阱）。
- 关键字乱序：关键字参数按名字匹配，顺序无关；可以用 `.keys()/values()` 遍历 `args` 字典。
- `list()` 的必要性：`args.keys()` 返回视图对象，没有下标能力，必须 `list()` 后才能 `[0]`、`[1:]`。
- 结合：`adder1(args.values())` 展示了字典的键值解包再传给参数收集——两套“打包/解包”机制互相衔接。

·常见误区：默认参数要用不可变值；args 键是字符串名，即函数的形参名。

题 5 与题 6. Dictionary tools (字典工具)

练习重点

- 实现自己的 copyDict(d) (复制字典) 与 addDict(d1,d2) (合并字典)。
- 比较"手写循环"与现代工具：D.copy()、D1.update(D2)、{D1, D2}、D1 | D2 (合并字典的四种方式)。
- 记住：e = d 不是复制，而是让 e 成为与 d 共享同一个字典对象的引用——改 d 会改 e。

代码 (dicttools.py 示例 B-9)

```
def copyDict(old):
    new = {}
    for key in old.keys():
        new[key] = old[key]      # —
    return new

def addDict(d1, d2):
    new = {}
    for key in d1.keys():
        new[key] = d1[key]
    for key in d2.keys():
        new[key] = d2[key]      # d2  d1
    return new
```

演示 (REPL)：

```
>>> from dicttools import *
>>> d = {1: 1, 2: 2}
>>> e = copyDict(d)      #
>>> d[2] = '?'           # d
>>> d                     # {1: 1, 2: '?'}
>>> e                     # {1: 1, 2: 2} — e
>>> x = {1: 1}; y = {2: 2}
>>> z = addDict(x, y)    # {1: 1, 2: 2}
```

深度理解

·核心概念：字典"拷贝"是浅拷贝——复制了键和里层引用，里层可变对象仍共享；`e = d` 则是别名（alias）而非拷贝。

·四种合并（Python 3.9+ 开始 | 运算符成了官方合并语法）：

1. 手写 `for` + 循环；
2. `D1.update(D2)`（就地修改 `D1`，返回 `None`）；
3. `{D1, D2}` 字典解包产生新字典；
4. `D1 | D2` 运算符合并，产生新字典。

·常见误区：把"引用两用"当拷贝用导致 JSON 编辑一改全改；`D.copy()` 是浅拷贝，深层 `json/deepcopy` 才处理嵌套。

题 7. More argument-matching（更多参数匹配）

目标文件

```
# testfuncs.py —
def f1(a, b): print(a, b)          #
def f2(a, *b): print(a, b)         # b
def f3(a, **b): print(a, b)        # b
def f4(a, *b, **c): print(a, b, c) #
def f5(a, b=2, c=3): print(a, b, c) #
def f6(a, b=2, *c): print(a, b, c) # +
```

REPL 交互（含解释注释）：

```
>>> from testfuncs import *
>>> f1(1, 2)          #          → 1 2
>>> f1(b=2, a=1)      #          → 1 2
>>> f2(1, 2, 3)       # 1(2,3) b → 1 (2, 3)
>>> f3(1, x=2, y=3)   # → 1 {'x': 2, 'y': 3}
>>> f4(1, 2, 3, **dict(x=2, y=3)) # → 1 (2, 3) {'x': 2, 'y...
>>> f5(1)             # → 1 2 3
>>> f5(1, 4)          # → 1 4 3
>>> f5(1, c=4)        # c → 1 2 4
>>> f6(1)             # → 1 2 ()
>>> f6(1, *[3, 4])    # → 1 3 (4,)
```

深度理解（参数匹配规则汇总）

·实参 → 形参的传递顺序：先位置匹配的非默认形参，再用关键字参数按名匹配，多余的位置进 name（元组）、多余的关键字进 name（字典），剩下默认值兜底。

·调用端的解包：func(iterable) 把可迭代拆成位置实参；func(mapping) 把字典拆成关键字实参——f6(1, [3,4]) 等价 f6(1, 3, 4)。

·设计思想：参数传递的"灵活的"让一个函数既能接收标准调用也能适配第三方"任意签名"的通用可复用接口。

·常见误区：在前在后是语法强制；默认参数不能放在无默认参数之前（`def f(a=1, b)` 是语法错误）；函数定义与调用端的 / 语义容易混淆。

题 8. Primes revisited (素数回归)

英文原文

Sample answer (file `primes.py`), with key notes: use `//` (floor division) to avoid fractional factors; wrapped the logic in a function + module so we can run it repeatedly; 加 `y <= 1` 处理边界。这是完整的交互与测试输出（略）。

中文翻译

这里练习的是"判断一个数是否是素数" ($y > 1$ 且只能被 1 和自身整除)。关键点：- 用 `//`（整除法）：`5 / 2` 会得到小数 2.5（但 2.5 不是上平=面上的"约数"），而 `5 // 2` 是 2，确保我们在整数范围检测。- 范围判定到 `y//2` 就够（更大约数一定更小），`while x > 1` 递减，凡整除即合数。- `break` 跳出循环后 `else` 不会执行：所以 `while ... else` 在没找到约数时打印 `is prime`。

完整代码 (示例 B-11 `primes.py`)

```
def prime(y):
    if y <= 1:                # 1
        print(y, 'is nonprime')
    else:
        x = y // 2            # // /
        while x > 1:
            if y % x == 0:     #
                print(y, 'has factor', x)
                break          #
            x -= 1
        else:                  # break→ →
            print(y, 'is prime')

tests = (27, 24, 13, 13.0, 15, 15.0, 3, 2, 1, -3)
for test in tests:
    prime(test)
```

```
$ python3 primes.py
27 has factor 9
24 has factor 12
13 is prime
13.0 is prime
15 has factor 5
15.0 has factor 5.0
3 is prime
2 is prime
1 is nonprime
-3 is nonprime
```

深度理解

- 算法本质：试除法 (trial division) ，从 $y//2$ 往下试除；有因子即合数，无因子即质数。
- 为什么用 `//` 与 `%`：`//` 是地板除、`%` 是取余。 $5.0 // 2 \rightarrow 2.0$ ， $13.0 // 2 \rightarrow 6.0$ ；连续 13.0 也判为质数（余数非零）。

- else 与 break 的搭配：循环 else 只在没有 break 时执行——这是本题最该学到关键结构。
- 脆弱点/改进方向：函数打印而非 return（可复用性差）；13.0 这种浮点会被当正式质数测（数学上应该非整数不考虑）；效率 $O(n)$ 不理想（可 `range(2, int(n0.5)+1)` 或 Miller-Rabin）。

题 9. Iterations and comprehensions (迭代与推导式)

原文（就是下四种写法）

```
>>> values = [2, 4, 9, 16, 25] >>> import math >>>
res = [] >>> for x in values: res.append(math.sqrt(x))
# 手动循环>>> res [1.4142135623730951, 2.0, 3.0, 4.0
, 5.0] >>> list(map(math.sqrt, values)) # map 内置函数
[1.4142135623730951, 2.0, 3.0, 4.0, 5.0] >>> [math.sq
rt(x) for x in values] # 列表推导式[1.414213562373095
1, 2.0, 3.0, 4.0, 5.0] >>> list(math.sqrt(x) for x in value
s) # 生成器表达式 (+list()) [1.4142135623730951, 2.0,
3.0, 4.0, 5.0]
```

中文要点

同任务四种写法结果一致，无“正确”之分。它们演示了过程式、函数式（map）、声明式（推导式）、惰性（生成器）四种编程风格；涉及算法选择。

深度理解

- map 参数顺序是"函数在前、可迭代在后"; 2.x 返回列表 (3.x 返回惰性迭代器, 需要 list()) 。
- 列表推导式更 Pythonic 且略快 (专门的 LISTAPPEND 字节码) 。
- 生成器表达式惰性求值, 避免中间大列表——大数据集内存友好。

题 10. Timing tools (计时工具)

(前置文件与思路)

- 示例 B-12: ../../Chapter21/timer2.py (Example 21-7) 的定义已列在示例包, 多用于时间戳、基准测试, 代码本附录不重打; 下方答案用 timer2.bestoftotal(test,...) 统计。
- 关键实现 (示例 B-13 timesqrt.py) 答点如下:

```
import sys
sys.path.append('../../Chapter21') # timer2.py
import timer2                       # 21 Part V

reps = 10_000
repslist = list(range(reps))        # range
from math import sqrt               # sqrt math.sqrt ...

def mathMod():
    for i in repslist:
        res = sqrt(i)               # mapsqrt
    return res
```

```

def powCall():
    for i in repslist:
        res = pow(i, .5)          # pow
    return res

def powExpr():
    for i in repslist:
        res = i ** .5            #
    return res

print(sys.version)
for test in (mathMod, powCall, powExpr):
    elapsed, result = timer2.bestoftotal(test, _reps1=5, _reps=100...)
    print(f'{test.__name__}: {elapsed:.5f} => {result}')

```

结果 (CPython 3.12, macOS) :

```

$ python3 timesqrt.py
3.12.2 [Clang 13.0.0 ...]
mathMod: 0.40860 => 99.99499987499375
powCall: 0.68245 => 99.99499987499375
powExpr: 0.57762 => 99.99499987499375

```

简评：函数 `sqrt` (C 库封装) 最快；运算符次之；`pow` 内置函数最慢。`pow` 需要做"指数参数多义性"处理，速度略逊。(PyPy 版本 8 倍 2 倍更快，因其 JIT 编译循环 + C 二级浮点。)

另：交互式比较字典推导式与 `for` 循环（同任务）的计时亦有说明（`dictcomp` 略快于一倍的 `for` 循环）。`bench` 结论因平台/版本而变——请按你的机器重测。

题 11. Recursive functions (递归函数)

```
def countdown(N):
    if N == 0:
        print('stop')          #
    else:
        print(N, end=' ')      #
        countdown(N - 1)       # N-1

# countdown(5) → 5 4 3 2 1 stop
# countdown(20) → 20 19 ... 2 1 stop
```

非递归一行同仁 (range) :

```
list(range(5, 0, -1))          # [5, 4, 3, 2, 1]
```

生成器递归版:

```
def countdown2(N):              #
    if N == 0:
        yield 'stop'
    else:
        yield N
        for x in countdown2(N - 1):
            yield x              # yield from countdown2(N-1)

list(countdown2(5))             # [5, 4, 3, 2, 1, 'stop']
```

深度理解

·递归基 case ($N=0$) 终止, 否则每次调用处理 N 、再调用 $N-1$ 的副本——每次递归都有独立栈帧, 局部 N 互不干扰。

·生成器的坑: 在生成器生成器里直接 `yield countdown2(N-1)` 只是返回一个生成器对象, 不产出它的元素; 必须

for ... : yield 或 yield from 代理。

·计数递减用递归"过度"了：range 更简单直接——这是个"何时该用递归（树型分解）、何时不该（顺序计数）"的绝佳案例。

题 12. Computing factorials (计算阶乘)

```
from functools import reduce
from timeit import repeat
import math

def fact0(N):
    #
    if N == 1:
        return N
    else:
        return N * fact0(N - 1)

def fact1(N):
    #
    return N if N == 1 else N * fact1(N - 1)

def fact2(N):
    # reduce
    return reduce(lambda x, y: x * y, range(1, N + 1))

def fact3(N):
    #
    res = 1
    for i in range(1, N + 1):
        res *= i
    return res

def fact4(N):
    #
    return math.factorial(N)

print(fact0(6), fact1(6), fact2(6), fact3(6), fact4(6))    # 6*5*4...
print(fact0(500) == fact1(500) == fact2(500) == fact3(500) == fact...
```

```
for test in (fact0, fact1, fact2, fact3, fact4):  
    print(test.__name__, min(repeat(stmt=lambda: test(500), number...
```

结果 (CP Python 3.12 / macOS) :

```
$ python3 factorials.py  
720 720 720 720 720  
True  
fact0 0.08720566902775317  
fact1 0.08635473699541762  
fact2 0.06704489700496197  
fact3 0.05152307112302461  
fact4 0.00873392098583281
```

深度理解

- 结论：递归 = 慢；迭代 = 中；标准库 `math.factorial` C 实现一骑绝尘。
- 递归最慢的原因：每次函数调用建立新栈帧、逐层返回乘法；且 N 到约 1000 触发 `RecursionError`。
- `reduce(lambda x, y: xy, range(1, N+1))` 中间的 `lambda` 每次调用都有开销。
- 教材意义：能"标准库 + 内置原语"就不要手写复杂逻辑——`math.factorial` 是答案作者推荐的"batteries"（电池）。
- 微基准（microbenchmark）结果随解释器/平台变化，别把单个数字当神谕。

Part V: Modules and Packages (模块与包)

英文原文

Part V, Modules and Packages See "Test Your Knowledge: Part V Exercises" in Chapter 25 for the exercises.

中文翻译

第五部分：模块与包。练习见第 25 章的 "Test Your Knowledge: Part V Exercises"。

深度理解

- 核心概念：Part V 练习围绕模块文件的组织——导入 (import/from)、自测 (main)、嵌套导入、包 (package)、重载 (reload)、循环导入 (circular imports)。
- 练习主线：以一个统计文件行数/字符数的小模块 `mymod.py` 为实验对象，反复改造它，逐步掌握模块系统的全部机制。

题 1. Import basics (导入基础)

英文原文

1. Import basics: When you're done, your file and REPL interaction with it should look similar to Example B-

16. Remember that Python can read a whole file into a list of line strings, and the len built-in returns the lengths of strings and lists: Example B-16. Part5/mymod.py (initial code, mymodstart.py)

```
python def countLines(name): file = open(name) return len(file.readlines()) def countChars(name): return len(open(name).read()) def test(name): # Or pass file object return countLines(name), countChars(name) # Or return a dictionary $ python3 >> import mymod >> mymod.test('mymod.py') (10, 281)
```

Your counts may vary for comments, an extra line at the end, and so on, and you don't need to set PYTHONPATH if the module is in the automatically searched current working directory. Note that these functions load the entire file in memory all at once, so they won't work for pathologically large files that are too big for your device's memory.

To be more robust, you could read line by line with iterators instead and count as you go (see Part5/mymodlines.py in the examples package):

```
python def countLines(name): tot = 0 for line in open(name): tot += 1 return tot def countChars(name): tot = 0 for line in open(name): tot += len(line) return tot A generator expression can have the same effect (though the excessive magic may cost you some points):
```

```
python def countLines(name): return sum(+1 for line
in open(name)) def countChars(name): return sum(le
n(line) for line in open(name))
```

 On Unix, you can verif
y your output with a `wc` command; on Windows, righ
t-click on your file to view its properties. Note that y
our script may report fewer characters than Windows
does—for portability, Python converts Windows `\r\n` l
ine-end markers to `\n`, thereby dropping one byte (c
haracter) per line.

To match byte counts with Windows exactly, you mus
t open in binary mode ('rb') or add the number of byt
es corresponding to the number of lines. See Chapter
s 9 and 37 for more on end-of-line translations in tex
t files.

The "ambitious" part of this exercise (passing in a file
object so you only open the file once) will require yo
u to use the `seek` method of the built-in file object. It
works like C's `fseek` call (and may call it behind the sc
enes): `seek` resets the current position in the file to a
passed-in offset. After a `seek`, future input/output op
erations are relative to the new position.

To rewind to the start of a file without closing and re
opening it, call `file.seek(0)`; the file read methods all p
ick up at the current position in the file, so you need
to rewind to reread.

Example B-17. `Part5/mymod2.py`

```
python def countLines(file): file.seek(0) # Rewind to start of file
return len(file.readlines()) def countChars(file): file.seek(0) # Ditto (rewind if needed)
return len(file.read()) def test(name): file = open(name) # Pass file object
return countLines(file), countChars(file) # Open file only once
$ python3 >> import mymod2 >> mymod2.test('mymod2.py') (12, 414)
```

中文翻译

1. 导入基础：完成之后，你的文件和与之交互的 REPL 会话应该类似于示例 B-16。记住：Python 可以把整个文件读入一个“行字符串列表”，而内置函数 `len` 返回字符串和列表的长度：

```
python def countLines(name): file = open(name)
return len(file.readlines()) # 行数 = 行列表的长度
def countChars(name): return len(open(name).read()) # 字符数 = 整个文件字符串的长度
def test(name): # 或者传入文件对象
return countLines(name), countChars(name) # 或者返回一个字典
>>> import mymod >>> mymod.test('mymod.py') (10, 281)
```

由于注释、末尾多一个空行等原因，你的计数可能有所不同；如果模块位于自动搜索的当前工作目录中，你不需要设置 `PYTHONPATH`。注意这些函数一次把整个文件读入内存，因此对“大到超出设备内存”的病态大文件无法工作。更健壮的做法是用迭代器逐行读取、边读边计数（见示例包的 `Part5/mymodlines.py`）：

```
python def countLines(name): tot = 0
for line in open(name): tot += 1
```

```

en(name): tot += 1 # 逐行迭代，边读边计
return tot
def countChars(name): tot = 0
for line in open(name): tot += len(line)
return tot

```

生成器表达式也能达到同样效果（不过过于“魔法”可能扣点分）：

```

python
def countLines(name): return sum(+1 for line in open(name))
def countChars(name): return sum(len(line) for line in open(name))

```

在 Unix 上，你可以用 `wc` 命令验证输出；在 Windows 上，右键文件查看属性即可。注意：你的脚本报出的字符数可能比 Windows 少——为了可移植性，Python 会把 Windows 的行结束符 `\r\n` 转换为 `\n`，因此每行会少一个字节（字符）。要与 Windows 的字节数完全一致，你必须用二进制模式（`'rb'`）打开，或加上与行数对应的字节数。

关于文本文件的行尾转换，参见第 9 章和第 37 章。本练习的“进阶”部分（传入文件对象、只打开一次文件）需要使用内置文件对象的 `seek` 方法。它像 C 的 `fseek` 调用一样工作（并可能在幕后调用它）：`seek` 把文件中的当前位置重置为传入的偏移量。`seek` 之后，后续输入/输出操作都相对新位置进行。要回到文件开头而不关闭重开，调用 `file.seek(0)`；文件 `read` 方法都从当前位置继续，所以要重读必须回卷。

```

python
def countLines(file): file.seek(0) # 回卷到文件开头
return len(file.readlines())
def countChars(file): file.seek(0) # 同样（需要时回卷）
return len(file.read())
def test(name): file = open(name) # 传入文件对象
return countLines(file), countChars(file) # 只打开一次文件
>>> import mymod2
>>> mymod2.test('mymod2.py')
(12, 414)

```

代码分析（逐行解读）

```
def countLines(name):
    file = open(name)           # 'r'
    return len(file.readlines()) # readlines() len

def countChars(name):
    return len(open(name).read()) # read() len

#
def countLines(file):
    file.seek(0)                # readlines ...
    return len(file.readlines())

def countChars(file):
    file.seek(0)                #
    return len(file.read())

def test(name):
    file = open(name)           #
    return countLines(file), countChars(file) #
```

深度理解

- 核心概念：文件对象是带位置指针的流——read/readline/readlines 都从"当前位置"开始。要多次读取同一文件，必须 seek(0) 回卷。
- 底层机制：seek 对应 C 的 fseek；文件指针（file position indicator）是操作系统层面的概念。read() 读全量、readlines() 读成行列表、readline() 读一行。
- 健壮性：一次读入内存的方式不适合超大文件；逐行迭代（文件对象本身可迭代）内存友好；sum(+1 for line in open(name)) 一行等价但可读性差。

- Windows 换行差异：Python 文本模式默认做通用换行（universal newlines）翻译： $\backslash r \backslash n \rightarrow \backslash n$ ，所以字节数会比 Windows 资源管理器显示的少；要精确字节数需 'rb' 二进制模式。
- 常见误区：忘了 seek 导致第二次计数为 0；用 open(name) 一次只读；把行数与字符数搞混（readlines 带 $\backslash n$ 也算一个字符）。

题 2. from / from （导入特定名）

英文原文

2. from/from\ : Here's the from part; replace with countChars to do the rest: \$ python3 >>> from mymod import >>> countChars('mymod.py') 281

中文翻译

2. from / from\ : 下面是 from 的部分；把换成 countChars 就能完成其余部分：>>> from mymod import >>> countChars('mymod.py') # 导入后无需模块名前缀，直接调用281

代码分析

```
from mymod import *           # mymod _
countChars('mymod.py')       # mymod.countChars(...)
# 281
```

深度理解

- 核心概念：from module import 把模块的公共名字（不含下划线开头的）批量复制到当前作用域——少打字，但也污染命名空间。
- 与 import module 的区别：import 引入的是模块对象本身（mymod.countChars）；from 引入的是模块里的名字（countChars）。两者都让名字指向同一函数对象。
- 最佳实践：模块可定义 all = ['名字', ...] 白名单控制导出；Python 官方更推荐显式 from module import 名字而非。
- 常见误区：from ... import 不会把下划线名字带进来；两个模块同名时会静默覆盖。

题 3. main（主模块自测）

英文原文

3. \\main\\: If you code it properly, this file works in either mode—program run or module import, as Example B-18 and the REPL session following it demo: Example B-18. Part5/mymod.py (edited)

```
python def countLines(name): file = open(name) return len(file.readlines()) def countChars(name): return len(open(name).read()) def test(name): # Or pass file object return countLines(name), countChars(name) #
```

Or return a dictionary if name == 'main': # Added: self-test code print(test('mymod.py')) # When run, not when imported \$ python3 mymod.py (13, 434)

This is where you would probably begin to consider using command-line arguments or user input to provide the filename to be counted instead of hardcoding it in the script. Examples B-19 and B-20 show the required mods (see Chapters 21 and 25 for more on sys.argv, and Chapter 10 for more on input): Example B-19. Part5/mymodargv.py (changed parts)

```
python if name == 'main': import sys # Command-line argument print(test(sys.argv[1])) $ python3 mymodargv.py mymod.py (13, 434)
```

Example B-20. Part5/mymodinput.py (changed parts)

```
python if name == 'main': print(test(input('Enter file name:'))) # Console/user input $ python3 mymodinput.py Enter file name: mymod.py (13, 434)
```

中文翻译

3. `\\main\\`: 只要编码得当, 这个文件在两种模式下都能工作——作为程序运行或作为模块导入, 如示例 B-18 及其后的 REPL 会话所示: `python if name == 'main':` # 新增: 自测代码 `print(test('mymod.py'))` # 只在“被运行”时执行, 导入时不执行 `$ python3 mymod.py` (13, 434) 到了这一步, 你可能开始考虑用命令行参数或用户输入来提供要

计数的文件名，而不是在脚本里写死。示例 B-19 和 B-20 展示了所需的改动（sys.argv 更多见第 21、25 章；input 更多见第 10 章）：

```
python # mymodargv.py 改动部分
if name == 'main': import sys # 命令行参数
print(test(sys.argv[1]))
$ python3 mymodargv.py mymod.py (13, 43)
python # mymodinput.py 改动部分
if name == 'main': print(test(input('Enter file name:'))) # 控制台/用户输入
$ python3 mymodinput.py Enter file name: mymod.py (13, 434)
```

代码分析（逐行解读）

```
if __name__ == '__main__':          # __name__
                                     # '__main__' import ...
    print(test('mymod.py'))         # —

# argv
import sys                          # sys.argv[0] sys.argv[1] ...
print(test(sys.argv[1]))            # python3 mymod_argv.py mymod.py

# input
print(test(input('Enter file name: '))) # input()
```

深度理解

·核心概念：if name == 'main': 是 Python 的自测（self-test）惯例——同一文件既可当程序跑（执行自测代码），又可当库导入（安静）。这是 Python 模块设计的精华之一。

·底层机制：解释器执行脚本时把该模块命名为 main；import 时命名空间字典里的 name 是模块的导入名。因此判断 name 就能区分两种运行场景。

·三个输入来源：写死（简单）、`sys.argv`（命令行，`sys.argv[0]` 是脚本路径，参数从 1 开始）、`input()`（交互式询问，返回字符串）。

·常见误区：忘了判断直接打印测试输出——导入时也会执行，造成“副作用”；`sys.argv[1]` 在没有参数时越界报 `IndexError`（应加判断或 `try`）。

题 4. Nested imports（嵌套导入）

英文原文

4. Nested imports: It's not much, but Example B-21 gives one solution and its results (the point here is to experiment with importing one module from another in a variety of ways): Example B-21. Part5/myclient.py

```
python from mymod import countLines, countChars
print(countLines('mymod.py'), countChars('mymod.py'))
```

\$ python3 myclient.py 13 434

As for the rest of this question, mymod's functions are accessible (that is, importable) from the top level of myclient, since from simply assigns to names in the importer (it works as if mymod's defs appeared in my client). For example, another file can say:

```
python import myclient myclient.countLines(...)
from myclient import countChars countChars(...)
```

If myclient used import instead of from, you'd need to use a path to get to the functions in mymod through myclient:

```
python import myclient myclient.mymod.countLines(..)
```

```
from myclient import mymod mymod.countChars(...)
```

In general, you can define collector modules that import all the names from other modules so they're available in a single convenience module. The following hypothetical code, for example, creates three different copies of the name somename—mod1.somename, collector.somename, and main.somename; all three share the same integer object initially, and only the name somename exists at the interactive prompt as is:

```
python # File mod1.py (hypothetical) somename = 99
# File collector.py (hypothetical) from mod1 import
# Collect lots of names here from mod2 import # "from"
# assigns to my names from mod3 import >> from
collector import somename
```

中文翻译

4. 嵌套导入：内容不多，示例 B-21 给出一个方案和它的结果（这里的重点是以多种方式实验“从一个模块导入另一个模块”）：
python # myclient.py from mymod import countLines, countChars # 导入特定名字
print(countLines('mymod.py'), countChars('mymod.py'))
\$ python3 m

yclient.py 13 434 至于本问题其余部分：mymod 的函数可以从 myclient 的顶层访问（即可导入），因为 from 只是向导入方命名空间中的名字赋值（就像 mymod 的 def 语句出现在 myclient 里一样）。例如，另一个文件可以这样写：python import myclient myclient.countLines(...) # 通过 myclient 访问 from myclient import countChars countChars(...) # 或者把名字再导入进来如果 myclient 用的是 import 而不是 from，你就需要通过 myclient 用路径访问 mymod 里的函数：python import myclient myclient.mymod.countLines(...) # 路径式访问：myclient 模块里还有 mymod 这个属性 from myclient import mymod mymod.countChars(...) 一般来说，你可以定义收集器（collector）模块，把其他模块的所有名字导入进来，使它们集中出现在一个便利模块中。例如下面这段假设代码会创建 somename 这个名字的三个不同副本——mod1.somename、collector.somename 和 main.somename；三者起初共享同一个整数对象，交互提示符下直接存在的只有 somename 这个名字：python # 文件 mod1.py（假设） somename = 99 # 文件 collector.py（假设） from mod1 import # 在这里收集大量名字 from mod2 import # "from" 把名字赋给我 from mod3 import >>> from collector import somename

代码分析（逐行解读）

```

from mymod import countLines, countChars    # from mymod ...
print(countLines('mymod.py'), countChars('mymod.py'))    #

#
import myclient
myclient.countLines(...)    # myclient  countLinesfrom ...
from myclient import countChars #

# myclient  import mymod
import myclient
myclient.mymod.countLines(...)    # myclient  → mymod  →...

```

深度理解

- 核心概念：import 引入"模块对象"作为属性；from 把"模块内名字"复制进当前命名空间。这决定了嵌套导入后访问路径的差异。
- 命名空间的本质：模块就是名字到对象的字典；from ... import ... 就是"复制字典条目"。因此 mymod 的函数会"像长在 myclient 里"一样可用。
- 收集器模式：聚合模块（collector）把多个模块的公共接口集中到一处，方便统一导入——大型库常这么做（如 tkinter、matplotlib.pyplot 的便利导出）。
- 同一对象多名字：mod1.somename、collector.somename、main.somename 是三个名字，但指向同一个整数对象——"名字 ≠ 对象"的心智模型再次体现。
- 常见误区：误以为 from 会复制对象（复制的是引用/名字）；在导入方修改 from 来的名字不会改原模块（名字重绑定）。

题 5. Package imports (包导入)

英文原文

5. Package imports: For this, copy the `mymod.py` solution file listed for exercise 3 (Example B-18) into a directory package. The following commands run in a Unix console set up the directory and an optional `init.py` file; you'll need to interpolate for other platforms and tools (e.g., use `copy` and `notepad` on Windows instead of `cp` and `vi`). This works in any directory, and you can do some of this from a file-explorer GUI, too.

When finished, you'll have a `mypkg` subdirectory that contains the files `init.py` and `mymod.py`. Technically, `mypkg` is located in the "home" directory component of the module search path. Notice how a `print` statement coded in the directory's initialization file fires only the first time it is imported, not the second. Raw strings (`r'...'`) can also avoid `\` escape issues in the file paths if you're working on Windows, but `/` works there too:

```
$ mkdir mypkg # Windows: same $ cp mymod.py mypkg/mymod.py # Windows: copy mymod.py mypkg\mymod.py $ vi mypkg/init.py # Windows: notepad mypkg\init.py...code a print statement...
```

```
$ python3 # Windows: py -3 (probably) >> import mypkg.mymod initializing mypkg >> mypkg.mymod.co
```

```
untLines('mypkg/mymod.py') # Windows: same 13 >
> from mypkg.mymod import countChars >> countC
hars('mypkg/mymod.py') # Windows: same 434
```

If you copy the module to main.py, the copy will run if you run the directory as a whole (though there may be no reason to do so in practice, as the original module can be run directly too):

```
$ cp mypkg/mymod.py mypkg/main.py # Windows:
copy $ python3 mypkg (13, 434)
$ python3 mypkg/mymod.py (13, 434)
```

中文翻译

5. 包导入：这个练习需要把练习 3 的 mymod.py 方案文件（示例 B-18）复制到一个目录包里。下面的命令在 Unix 控制台中运行，用来建立目录和一个可选的 init.py 文件；其他平台和工具需要自行替换（例如 Windows 上用 copy 和 notepad 代替 cp 和 vi）。这在任何目录里都成立，你也可以在文件资源管理器 GUI 里完成其中一部分。完成后，你会有一个包含 init.py 和 mymod.py 的 mypkg 子目录。从技术上讲，mypkg 位于模块搜索路径的"home"目录部分。注意：在目录初始化文件中编写的 print 语句只在第一次导入时触发，第二次不会。在 Windows 上工作的话，原始字符串 (r'...') 也可以避免文件路径中的 \ 转义问题，不过那里用 / 也可以：\$ mkdir mypkg # Windows: 相同\$ cp mymod.py mypkg/mymod.py # Windows: copy mymod.py mypkg\mymod.py \$ vi mypkg/init.py #

Windows: notepad mypkg\init.py...在文件中编写一条 print 语句...\$ python3 # Windows: 用 py -3 (通常) >>> import mypkg.mymod initializing mypkg # 第一次导入: init.py 被执行>>> mypkg.mymod.countLines('mypkg/mymod.py') # Windows: 相同13 >>> from mypkg.mymod import countChars >>> countChars('mypkg/mymod.py') # Windows: 相同434 # 第二次导入: 没有再次打印 initializing 如果把模块复制成 main.py, 那么以目录整体运行时执行这份副本 (虽然实践中可能没理由这样做, 因为原模块也可以直接运行): \$ cp mypkg/mymod.py mypkg/main.py # Windows: copy \$ python3 mypkg # 运行整个目录 → 执行 main.py (13, 434) \$ python3 mypkg/mymod.py # 或直接运行模块文件(13, 434)

代码分析 (逐行解读)

```
mkdir mypkg #
cp mymod.py mypkg/mymod.py #
vi mypkg/__init__.py # print('initializi...
python3
>>> import mypkg.mymod # import mypkg → __init__...
# import mypkg.mymod → mym...
>>> mypkg.mymod.countLines('mypkg/mymod.py') # ..
>>> from mypkg.mymod import countChars #
```

深度理解

·核心概念: 包 (package) = 含 init.py 的目录。import mypkg.mymod 会先执行 init.py (初始化包), 再执行 mymod.py。

·为什么 initializing 只打印一次: 模块与包都缓存进 sys.

modules, 第二次 from mypkg.mymod import ... 直接取缓存, init.py 不再执行——这与 Part I 练习 3 的模块缓存是同一个机制。

- init.py 的职责: 包级初始化、all 导出控制、把子模块名字上提 (from .mymod import countLines 等)。

- main.py: 目录作为整体被 python3 mypkg 运行时执行的入口文件。

- Windows 路径: \ 在普通字符串里是转义符, 用 r'C:\...' 或 / (Windows 也接受正斜杠)。

- 常见误区: 忘记创建 init.py (3.3+ 的命名空间包除外); 以为 import mypkg.mymod 后可以直接写 countLines (还需 from 或属性路径); 以为每次 import 都重跑包初始化。

题 6. Reload (重载)

英文原文

6. Reloads: This exercise just asks you to experiment with changing the changer.py example in the book's Example 23-10, so there's nothing to show here.

中文翻译

6. 重载: 这个练习只是让你实验性地修改书中示例 23-10 的 changer.py 例子, 所以这里没有需要展示的内容。

深度理解

·核心概念: `importlib.reload(module)` (旧版是内置 `reload`) 重新执行模块代码——开发期"改完代码立即在 REPL 里生效"的利器。

·要点: `reload` 重跑的是模块体, 已有引用旧对象的名字不会自动更新; `from ... import` 名字与 `reload` 混用是经典的"改了个寂寞"陷阱。

·最佳实践: 改动模块后, 要么 `reload` 该模块, 要么重启解释器; 使用 `from` 导入的名字需重新执行 `from`。

题 7. Circular imports (循环导入)

英文原文

7. Circular imports: The short story is that importing `recur2` first works because the recursive import then happens at the import in `recur1`, not at a `from` in `recur2`. The long story goes like this: importing `recur2` first works because the recursive import from `recur1` to `recur2` fetches `recur2` as a whole instead of getting specific names. `recur2` is incomplete when it's imported from `recur1`, but because it uses `import` instead of `from`, you're safe: Python finds and returns the already created `recur2` module object and continues to run the rest of `recur1` without a glitch. When the `recur2` import resumes, the second `from` finds the name `Y` in `recur1` (it's been run completely), so no error is reported.

d. Running a file as a script is not the same as importing it as a module; these cases are the same as running the first import or from in the script interactively. For instance, running recur1 as a script works because it is the same as importing recur2 interactively, as recur2 is the first module imported in recur1. Running recur2 as a script fails for the same reason—it's the same as running its first import interactively.

中文翻译

7. 循环导入：简短版：先导入 recur2 能成功，是因为递归导入发生在 recur1 里的 import 处，而不是 recur2 里的 from 处。详细版是这样的：先导入 recur2 能成功，是因为从 recur1 到 recur2 的递归导入是整体获取 recur2 模块对象，而不是获取特定名字。recur2 在被 recur1 导入时是不完整的，但因为它用的是 import 而不是 from，所以你是安全的：Python 找到并返回已创建的 recur2 模块对象，继续毫无障碍地运行 recur1 的其余部分。当 recur2 的导入恢复执行时，第二个 from 能在 recur1 中找到名字 Y（它已完整运行），因此不会报错。把文件作为脚本运行与把它作为模块导入不是一回事；这些情况等同于在交互模式下运行脚本中的第一条 import 或 from。例如，把 recur1 当脚本运行可以成功，因为它等同于交互式导入 recur2——因为 recur2 是 recur1 中第一个被导入的模块。把 recur2 当脚本运行失败也是同样的原因——它等同于交互式执行它的第一条导入语句。

深度理解

- 核心概念：循环导入（A 导 B、B 导 A）的成败取决于导入时刻的模块完成度与导入方式。
- 底层机制：import 的第一步是"找到模块 → 创建模块对象 → 放入 sys.modules → 执行模块体"。递归发生时，Python 会在 sys.modules 里发现"半成品"模块对象并直接返回它（不再重复执行）——所以 import 整体导入安全；而 from recur1 import Y 在 B 的顶层立刻取值，若此时 recur1 还没执行到定义 Y 的那行，就会 ImportError/AttributeError。
- 为什么脚本运行与导入不同：脚本运行 = 模块以 main 身份从头执行；谁先导入谁决定了顺序与成败。
- 实战经验：设计模块时避免循环依赖；实在需要时——把 from 挪进函数体（延迟到调用时）、或用 import module + module.attr 形式访问。
- 常见误区：以为"循环导入总是错误"——它只在"时机不巧 + 用 from"时才会爆雷；同理，REPL 里手动按序导入往往掩盖了问题。

Part VI: Classes and OOP (类与面向对象)

英文原文

Part VI, Classes and OOP See "Test Your Knowledge: Part VI Exercises" in Chapter 32 for the exercises.

中文翻译

第六部分：类与面向对象。练习见第 32 章的 "Test Your Knowledge: Part VI Exercises"。

深度理解

·核心概念：Part VI 练习覆盖继承、运算符重载、子类化、属性拦截方法、包装/代理类 (wrapper)、类树 (class tree)、组合 (composition) 与多态分派。

·练习主线：亲手实现"加法家族" (Adder)、"列表包装器" (MyList)、"集合包装器" (Set/MultiSet)、"消息路由" (Lunch)、"动物分类树" (zoo) ——每一个都是真实设计模式的迷你版。

题 1. Inheritance (继承)

英文原文

1. Inheritance: Example B-22 lists a solution for this exercise, along with some interactive tests. The add overload has to appear only once, in the superclass, as it invokes type-specific add methods in subclasses: Example B-22. Part6/adder.py

```
python class Adder: def add(self, x, y): print('not implemented!') def init(self, start=[]): self.data = start def add(self, other): # Or in subclasses? return self.add(self.data, other) # Or return type? class ListAdder(Adder): def add(self, x, y): return x + y class DictAdder(Adder): def add(self, x, y): new = {} for k in x.keys(): new[k] = x[k] for k in y.keys(): new[k] = y[k] return new $ python3 >> from adder import >> x = Adder() >> x.add(1, 2) not implemented!
```

```
>> x = ListAdder() >> x.add([1], [2]) [1, 2] >> x = DictAdder() >> x.add({1: 1}, {2: 2}) {1: 1, 2: 2} >> x = Adder([1]) >> x + [2] not implemented!
```

```
>> >> x = ListAdder([1]) >> x + [2] [1, 2] >> [2] + x
TypeError: can only concatenate list (not "ListAdder") to list
```

Notice in the last test that you get an error for expressions where a class instance appears on the right of a +; if you want to fix this, use radd methods, as described in Chapter 30.

If you are saving a value in the instance anyhow, you might as well rewrite the add method to take just one argument, in the spirit of other examples in this part of the book. Example B-23 sketches this mutation: Example B-23. Part6/adder2.py

```
python class Adder: def init(self, start=[]): self.data = start def add(self, other): # Pass a single argument return self.add(other) # The left side is in self def add(s
```

```
elf, y): print('not implemented!') class ListAdder(Adder): def add(self, y): return self.data + y class DictAdder(Adder): def add(self, y): d = self.data.copy() # Change to use self.data instead of x d.update(y) # Or "cheat" by using quicker built-ins
```

```
return d x = ListAdder([1, 2, 3]) y = x + [4, 5, 6] print(y) # Prints [1, 2, 3, 4, 5, 6] z = DictAdder(dict(name='x')) + {'a': 1} print(z) # Prints {'name':'x','a': 1} Because values are attached to objects rather than passed around, this version is arguably more object-oriented. And, once you've gotten to this point, you'll probably find that you can get rid of add altogether and simply define type-specific add methods in the two subclasses.
```

中文翻译

1. 继承：示例 B-22 给出了本练习的一个方案，并附上一些交互式测试。add 重载只需要在超类中出现一次，因为它调用的是子类中类型特有的 add 方法：python

```
class Adder: # 超类：定义"骨架" def add(self, x, y): print('not implemented!') # 占位实现——子类必须覆写 def init(self, start=[]): self.data = start def add(self, other): # 或者在子类中定义？ return self.add(self.data, other) # 或者返回类型？ class ListAdder(Adder): def add(self, x, y): return x + y # 列表拼接 class DictAdder(Adder): def add(self, x, y): new = {} for k in x.keys(): new[k] = x[k] # 逐键复制 x for k in y.keys(): new[k] = y[k] # 再复制 y (键重复则 y 覆盖) return new >>> x = Adder() >>> x.ad
```

`d(1, 2) → not implemented!` (基类占位实现) `>>> x = ListAdder()` `>>> x.add([1], [2]) → [1, 2]` (子类覆写后生效) `>>> x = DictAdder()` `>>> x.add({1: 1}, {2: 2}) → {1: 1, 2: 2}` `>>> x = Adder([1])` `>>> x + [2] → not implemented!` (基类 `add` 未覆写) `>>> x = ListAdder([1])` `>>> x + [2] → [1, 2]` (`add` 调 `self.add` → 子类实现) `>>> [2] + x → TypeError` (实例在 `+` 右侧, `list` 不认识它) 注意最后一个测试: 当类实例出现在 `+` 的右侧时你会得到错误; 想修复它, 就按第 30 章的说明使用 `radd` 方法。反正你都要在实例中保存值, 那不如把 `add` 方法改写成只接收一个参数, 与本书该部分的其他示例风格一致。示例 B-23 勾勒了这个改造:

```
python class Adder:
    def init(self, start=[]):
        self.data = start
    def add(self, other):
        # 只传一个参数
        return self.add(other)
# 左侧对象在 self 里
def add(self, y):
    print('not implemented!')
class ListAdder(Adder):
    def add(self, y):
        return self.data + y
# 状态存于 self.data
class DictAdder(Adder):
    def add(self, y):
        d = self.data.copy()
        # 复制自身数据 (浅拷贝)
        d.update(y)
        # 或"作弊"用更快的内置
        return d
x = ListAdder([1, 2, 3])
y = x + [4, 5, 6]
# 打印 [1, 2, 3, 4, 5, 6]
z = DictAdder(dict(name='x')) + {'a': 1}
# 打印 {'name': 'x', 'a': 1}
```

因为值被附着在对象上而不是传来传去, 这个版本可以说更符合面向对象精神。而且, 一旦到了这一步, 你可能会发现 `add` 方法可以整个删掉, 直接在两个子类里定义类型特异的 `add` 方法。

代码分析 (逐行解读)

```

class Adder:
    # = template meth...
    def add(self, x, y):
        print('not implemented!') # ""
    def __init__(self, start=[]): # []
        self.data = start
    def __add__(self, other): #
        return self.add(self.data, other) # self.add ...

class ListAdder(Adder):
    def add(self, x, y):
        return x + y #

class DictAdder(Adder):
    def add(self, x, y):
        new = {}
        for k in x.keys(): new[k] = x[k] # x
        for k in y.keys(): new[k] = y[k] # y
        return new

```

· $x + [2]$ 的求值链: $x.add([2]) \rightarrow$ (继承自 Adder) $\rightarrow self.add(self.data, [2]) \rightarrow$ 沿 MRO 找到 ListAdder.add $\rightarrow [1] + [2]$ 。

· $[2] + x$: list.add(x) 不认识 ListAdder $\rightarrow$ 抛 TypeError。补救: 定义 radd。

深度理解

· 核心概念: 继承 + 方法覆写 + 动态分派 (dynamic dispatch) 组成"模板方法"设计模式——骨架在超类定一次, 细节在子类各自实现。

· 底层机制: $self.add(...)$ 是属性查找: 沿 $self.class$ 的 MRO (方法解析顺序) 从子类向父类找 add。这就是"多态"在类层面的实现——同一行代码、不同对象、不同行为

- 运算符重载的放置：add 只需在超类定义一次，因为 self.add 是虚调用（virtual call），子类覆写 add 即可让 + 自动分派——少写代码、统一契约。
- 为什么 [2] + x 失败：+ 左侧优先使用左侧类型的 add；左侧是 list，它的 add 只接受 list。右侧补救用 radd (reflected addition, 反射加法)，它只在左侧类型拒绝时才被调用。
- 可变默认值陷阱：def init(self, start=[]) 中 [] 在定义时求值一次、被所有实例共享——这里只读（被重新赋值）问题不大，但若原地修改会“串味”。官方建议用 start=None 再在函数体内赋新列表。
- 版本演进：adder2.py 把状态收进 self.data、add 只收一个参数——“左操作数即 self”，更 OOP；作者甚至提示可以完全去掉 add，直接在各子类定义 add。
- 常见误区：忘了在子类覆写 add 导致静默输出 "not implemented!"；把运算符重载写在子类造成重复代码；忽略 radd 导致右侧运算崩。

题 2. Operator overloading (运算符重载：包装列表)

英文原文

2. Operator overloading: The solution code and its REPL results in Example B-24 demo a handful of operat

or-overloading methods we explored in Chapter 30. Copying the initial value in the constructor is important because it may be mutable; you don't want to change or have a reference to an object that's possibly shared somewhere outside the class. The `getattr` method routes calls to the wrapped list. For tips on a possibly easier way to code this, see "Extending Types by Subclassing" in Chapter 32: Example B-24. `Part6/mylist.py`

```
python class MyList: def init(self, start): #self.wrapped
= start[:] # Copy start: no side effects self.wrapped =
list(start) # Make sure it's a list here def add(self, oth
er): return MyList(self.wrapped + other) def mul(self,
time): return MyList(self.wrapped time) def getitem(s
elf, offset): # Also passed a slice on [:] return self.wra
pped[offset] # For iteration if no iter def len(self): ret
urn len(self.wrapped) # Also fallback for truth tests d
ef append(self, node): self.wrapped.append(node) def
getattr(self, name): # Other methods: sort/reverse/et
c. return getattr(self.wrapped, name) def repr(self): #
Catchall display method return repr(self.wrapped) if n
ame == 'main': x = MyList('hack') print(x) # ['h','a','c','
k'] print(x[2]) # c print(x[1:]) # ['a','c','k'] print(x + ['co
de']) # ['h','a','c','k','code'] print(x 3) # ['h','a','c','k'] 3 x
.append('1'); x.extend(['z']) # extend 走 getattr x.sort()
# sort 走 getattr
```

```
print(''.join(c for c in x)) # 迭代: 1 a c h k z $ python3
mylist.py ['h','a','c','k'] c ['a','c','k'] ['h','a','c','k','code'] [
```

'h','a','c','k','h','a','c','k','h','a','c','k'] 1 a c h k z

Note that it's also important to copy the start value by calling `list` instead of slicing here, because otherwise the result may not be a true list, and so will not respond to expected list methods, such as `append` (e.g., slicing a string returns another string, not a list). You would be able to copy a `MyList` start value by slicing because its class overloads the slicing operation and provides the expected list interface; however, you need to avoid slice-based copying for objects such as strings.

中文翻译

2. 运算符重载：示例 B-24 中的解决方案代码及其 REPL 结果演示了我们在第 30 章探讨过的一批运算符重载方法。在构造函数里复制初始值很重要，因为它可能是可变的——你不希望改动或引用一个可能在类外部某处被共享的对象。`getattr` 方法把调用路由到被包装的列表。关于更简单的实现提示，参见第 32 章的 "Extending Types by Subclassing"（通过子类化扩展类型）：`python class MyList: # 包装器 (wrapper) : 包裹一个真正 list`
`def init(self, start): #self.wrapped = start[:] # 用切片复制 start: 无副作用`
`self.wrapped = list(start) # 确保这里是个 list`
`def add(self, other): return MyList(self.wrapped + other) # 返回 MyList 保持链式`
`def mul(self, time): return MyList(self.wrapped time)`
`def getitem(self, offset): # 切片 [:] 也会传 slice 对象进来`
`return self.wrapped[offset] # 无 iter 时 for 循环也靠它`
`def len(self): return len(self.wrapped)`

```
# 真值测试的兜底def append(self, node): self.wrapped.append(node) # 显式代理def getattr(self, name): # 其余方法: sort/reverse 等都转发return getattr(self.wrapped, name) def repr(self): # 显示兜底方法return repr(self.wrapped) if name == 'main': # 自测代码x = MyList('hack') print(x) # ['h','a','c','k'] print(x[2]) # c print(x[1:]) # ['a','c','k'] print(x + ['code']) # ['h','a','c','k','code'] print(x * 3) # ['h','a','c','k'] 重复 3 次x.append('1'); x.extend(['z']) # extend 通过 getattr 转发x.sort() # sort 通过 getattr 转发print(''.join(c for c in x)) # 迭代 x → 1 a c h k z (sort 后顺序) $ python3 mylist.py ['h','a','c','k'] c ['a','c','k'] ['h','a','c','k','code'] ['h','a','c','k','h','a','c','k','h','a','c','k'] 1 a c h k z 注意: 这里用 list 而不是切片来复制 start 值也很重要, 否则结果可能不是真正的 list, 因此无法响应预期的列表方法 (比如 append) (例如, 切片一个字符串得到的是另一个字符串, 而不是列表)。对 MyList 的 start 值你倒是可以放心用切片复制, 因为它的类重载了切片操作、提供了预期的列表接口; 但对字符串这类对象, 你需要避免基于切片的复制。
```

代码分析 (逐行解读)

```
class MyList:                                # "/" list
    def __init__(self, start):
        self.wrapped = list(start)          # list() start → ...
                                                # start MyList→sta...

    def __add__(self, other):
        return MyList(self.wrapped + other) # + MyList...

    def __mul__(self, time):
        return MyList(self.wrapped * time)  # *

    def __getitem__(self, offset):
```

```

        return self.wrapped[offset] # offset sli...
def __len__(self):
    return len(self.wrapped) # len() bool
def append(self, node):
    self.wrapped.append(node) #
def __getattr__(self, name): #
    return getattr(self.wrapped, name) # /
def __repr__(self):
    return repr(self.wrapped) # /

#
x = MyList('hack') # list('hack') → ['h','a','c','k']
x[1:] # __getitem__(slice(1,None)) → ['a','c','k...
x + ['code'] # __add__ → MyList(['h','a','c','k','code'...
x * 3 # __mul__ → MyList( 3 )
x.extend(['z']) # MyList extend → __getattr__ → li...
x.sort() # sort ['1','a','c','h','k','z']
' '.join(c for c in x) # for __iter__ __getitem__(0,1...

```

深度理解

· 核心概念：包装 (wrapper) / 代理 (proxy) 模式——类持有一个内部对象，对外重定义核心操作，其余操作通过 `getattr` 全自动转发。这是 Python 实现“安全子类化之外的扩展”的经典手段。

· `getattr` 只兜“找不到”的属性：MyList 自己定义了 `append`、`add` 等，这些不会走 `getattr`；只有 `extend`、`sort`、`reverse` 等未定义的才转发。注意 `getattr` 对内建隐式操作（如 `+`、`len()` 不走它）——它们直接查类型槽，这也是本部分练习 4 的主题。

· 为什么 `list(start)` 而不是 `start[:]`：切片返回“同类型”——`'hack'[:]` 是字符串，没有 `append`；`list()` 保证结果一

定是 list (字符串→字符列表、元组→列表、列表→浅拷贝)。

·为什么构造函数要复制: start 可能是调用方仍在用的可变对象; 不复制的话内部与外部共享同一对象, 外部改动会"泄漏"进来。

·迭代的兜底机制: 类没有 iter 时, for x in obj 会退化为"反复 getitem(0)、[1]、[2]... 直到 IndexError"——这是 Python 的"迭代协议退路"。

·常见误区: 在 getattr 里访问自身不存在的属性造成无限递归; 忘了 repr 导致打印 <MyList object at ...>; 用切片复制字符串导致内部不是 list。

题 3. Subclassing (子类化: 带统计的 MyList Sub)

英文原文

3. Subclassing: One solution appears in Example B-25 ; your solution will be similar. You can also use super here instead of explicit superclass names for methods and attributes, as partly noted in the code's comments: Example B-25. Part6/mysub.py

```
python from mylist import MyList class MyListSub(MyList): calls = 0 # Shared by instances def init(self, start): self.adds = 0 # Varies in each instance MyList.init(self, start) # Or: super().init(start)
```

```

def add(self, other): print('add:' + str(other)) MyListSub.calls += 1 # Class-wide counter

self.adds += 1 # Per-instance counts return MyList.add(self, other) # Or: super().add(other) def stats(self): return self.calls, self.adds # All adds, my adds if name == 'main': x = MyListSub('read') y = MyListSub('code') print(x[2]) # e print(x[1:]) # ['e','a','d'] print(x + ['lp6e']) # add: ['lp6e'] ['r','e','a','d','lp6e'] print(x + ['book']) # add: ['book'] ... print(y + ['py312']) # add: ['py312'] .. print(x.stats()) # (3, 2) $ python3 mysub.py e ['e','a','d'] add: ['lp6e'] ['r','e','a','d','lp6e'] add: ['book'] ['r','e','a','d','book'] add: ['py312'] ['c','o','d','e','py312'] (3, 2)

```

中文翻译

3. 子类化：一种方案见示例 B-25；你的方案会与之类似。这里你也可以用 `super` 代替显式写出超类名来访问方法和属性，代码注释中部分提及：`python class MyListSub(MyList):` # 继承 `MyList` 的包装功能 `calls = 0` # 类变量：所有实例共享 `def init(self, start):` `self.adds = 0` # 实例变量：每个实例独立 `MyList.init(self, start)` # 或 `super().init(start)`：先初始化父类部分 `def add(self, other):` `print('add:' + str(other))` `MyListSub.calls += 1` # 类级计数器（所有实例共用一个） `self.adds += 1` # 实例级计数器 `return MyList.add(self, other)` # 或 `super().add(other)`：复用父类逻辑 `def stats(self):` `return self.calls, self.adds` # (总加法次数, 本实例加法次数) `>>> x = MyListSub('read')` `>>> x[2] → e` (继承自 `MyList` 的 `getitem`) `>>> x[1:] → ['e','`

a','d'] >>> x + ['lp6e'] → add: ['lp6e'] 然后 ['r','e','a','d','lp6e'] >>> x.stats() → (3, 2) (全局 3 次 +, x 自己 2 次)

代码分析（逐行解读）

```
class MyListSub(MyList):           # MyList
    calls = 0                       #
    def __init__(self, start):
        self.adds = 0              # self
        MyList.__init__(self, start) # super().__ini...
    def __add__(self, other):
        print('add: ' + str(other)) # +
        MyListSub.calls += 1        # + +1
        self.adds += 1              #
        return MyList.__add__(self, other) # __add__ ...
    def stats(self):
        return self.calls, self.adds # self.calls →...
```

深度理解

· 核心概念：类变量 vs 实例变量——calls 定义在类体（被所有实例共享，MyListSub.calls 与 x.calls 起初指向同一整数对象）；adds 在 init 里用 self.adds 创建（每实例一份）。属性查找顺序：实例字典 → 类 → 基类。

· += 与共享的微妙处：MyListSub.calls += 1 是“读类变量 → 加一 → 写回类”；self.adds += 1 是“读实例属性 → 写回实例属性”——因为 self.adds 在实例字典里已有，不会污染类。若对类变量用 self.calls += 1 则会在实例上创建新属性、遮蔽类变量（经典陷阱）。

· 委托父类：覆写 add 后显式 MyList.add(self, other)（或 super().add(other)）复用父类逻辑——“覆写 + 委托

"是子类化的标准姿势。

·为什么要调父类 init: 父类构造器负责设置 self.wrapped; 子类不调用它, 包装列表就不存在。

·输出分析: 三次 + (x 两次、y 一次) → calls=3; x 自身 adds=2 → stats() 返回 (3, 2)。

·常见误区: 忘记调用父类 init; 用 self.calls += 1 意外创建实例属性; 以为类变量"每个实例一份"。

题 4. Attribute methods (属性拦截方法)

英文原文

4. Attribute methods: The following works through this exercise. As noted in Chapter 28 and elsewhere, getattr is not called for built-in operations in Python 3.X, so the expressions aren't intercepted at all here; a class like this must somehow redefine X operator-overloading methods explicitly.

You can find more on this limitation in Chapters 28, 31, 32, and 38, as well as workarounds for it in Chapter 39 and its inheritance special case in Chapter 40. Its impacts are potentially broad but can be addressed with code:

```
$ python3 >> class Attrs:
... def getattr(self, name): ... print('get:', name) ... def setattr(self, name, value): ... print('set:', name, value) >
```

```
> x = Attrs() >> x.append get append >> x.lang = 'py
312'set: lang py312 >> x + 2 TypeError: unsupported
operand type(s) for +:'Attrs' and'int'>> x[1] TypeErro
r:'Attrs' object is not subscriptable >> x[1:5] TypeErro
r:'Attrs' object is not subscriptable
```

中文翻译

4. 属性方法：下面是对这个练习的完整演练。如第 28 章等处所述，在 Python 3.X 中，`getattr` 不会为内置运算（built-in operations）调用，所以这里的表达式完全不会被拦截；像这样的类必须显式重定义 `X` 形式的运算符重载方法。关于这个局限的更多内容见第 28、31、32、38 章，它的变通方案见第 39 章，以及它在第 40 章中的继承特例。其影响可能很广，但都可以用代码应对：

```
>>> class Attrs:
... def getattr(self, name): ... print('get:', name) # 拦截"
读不存在的属性" ... def setattr(self, name, value): ... pri
nt('set:', name, value) # 拦截"所有属性赋值" >>> x = A
ttrs() >>> x.append get append # 属性查找失败 → get
attr 被调用>>> x.lang = 'py312'set: lang py312 # 所有
赋值都经过 setattr >>> x + 2 TypeError: unsupported
operand type(s) for +:'Attrs' and'int' # 没定义 add, 也
没走 getattr >>> x[1] TypeError:'Attrs' object is not s
ubscriptable # 没定义getitem >>> x[1:5] TypeError:'A
ttrs' object is not subscriptable # 同样
```

代码分析（逐行解读）

```
class Attrs:
    def __getattr__(self, name):          # →→
        print('get:', name)
    def __setattr__(self, name, value): # self.name = value ...
        print('set:', name, value)

x = Attrs()
x.append          # 'append' // → __getattr__ → get: ...
x.lang = 'py312'  # → __setattr__ → set: lang py312
x + 2             # + __add__Attrs →
                  # TypeError__getattr__
x[1]              # __getitem__ → → TypeError
x[1:5]           #
```

深度理解

- 核心概念：Python 3.X 中属性拦截有两套独立机制——属性访问（`x.name`）走 `getattr/setattr`；内建运算符（`+`、`[]`、`len()` 等）走类型槽（slot，即 `add`、`getitem` 等）。两者互不相通。
- 为什么这样设计：性能考量——内建操作走 C 级快速路径（类型槽直接在类型对象上），不经过 Python 级的方法查找；若都走 `getattr` 会拖慢所有内建操作。
- 影响：像 `getattr` 那样的“万能代理类”无法靠它拦截 `+` 与索引——必须在类里显式定义 `add`、`getitem` 等。这是实现“全功能代理”（如 `RPyC`、`mock` 库）时绕不开的工作量，第 39 章给出一些变通。
- `setattr` 的递归陷阱：在 `setattr` 里写 `self.name = value` 会无限递归——必须用 `object.setattr(self, name, val`

ue) 或 `self.dict[name] = value`。

·常见误区：以为 `getattr` 能拦截一切；以为定义了 `getattr` 后 `+` 也会被打印。

题 5. Set objects (集合对象)

英文原文

5. Set objects: Here's the sort of interaction you should get. To make the import of `Chapter32/setwrapper.py` work, either run this in the folder where this file resides, copy this file to your working directory, or add this file's folder to your import search path per Part V. Comments explain which methods are called. Also, bear in mind that sets are a built-in type in Python, so this is mostly just a coding exercise (see Chapter 5 for more on sets):

```
$ python3 >> from setwrapper import Set >> x = Set([1, 2, 3, 4]) # Runs init >> y = Set([3, 4, 5]) >> x & y # and, intersect, then repr Set:[3, 4] >> x | y # or, union, then repr Set:[1, 2, 3, 4, 5] >> z = Set('hello') # init removes duplicates >> z[0], z[-1], z[2:] # getitem ('h', 'o', ['l', 'o']) >> for c in z: print(c, end='') # iter (else getitem) h e l o >> ''.join(c.upper() for c in z) # iter (else getitem)'HELO'>> len(z), z # len, repr (4, Set:['h','e','l','o']) >> z & 'mello', z | 'mello'(Set:['e','l','o'], Set:['h','e','l','o','m'])
```

A solution to the multiple-operand extension subclass looks like the class in Example B-26. It needs to replace only two methods in the original set. The class's documentation string explains how it works: Example B-26. Part6/multiset.py

```
python from setwrapper import Set class MultiSet(Set): """Inherits all Set names, but extends intersect and union to support multiple operands. Note that "self" is still the first argument (stored in the args argument now). Also note that the inherited & and | operators call the new methods here with 2 arguments, but processing more than 2 requires a method call, not an expression. intersect doesn't remove duplicates here: the Set constructor does.
```

```
"""
```

```
def intersect(self, others): res = [] for x in self: # Scan first sequence for other in others: # For all other args if x not in other: break # Item in each one? else: # No : break out of loop res.append(x) # Yes: add item to end
```

```
return Set(res) def union(args): # self is args[0] res = [ ] for seq in args: # For all args for x in seq: # For all nodes if not x in res: res.append(x) # Add new items to result
```

```
return Set(res) Your interaction with this extension will look something like the following. Note that you can intersect by using & or calling intersect, but you m
```

ust call intersect for three or more operands; & is a binary (two-sided) operator. Also, note that we could have called MultiSet simply Set to make this change more transparent if we used setwrapper.Set to refer to the original within multiset (the as clause in an import could rename the class too if desired):

```
>> from multiset import >> x = MultiSet([1, 2, 3, 4])
>> y = MultiSet([3, 4, 5]) >> z = MultiSet([0, 1, 2]) >
> x & y, x | y # Two operands (Set:[3, 4], Set:[1, 2, 3, 4
, 5]) >> x.intersect(y, z) # Three operands Set:[] >> x.
union(y, z) Set:[1, 2, 3, 4, 5, 0] >> x.intersect([1,2,3], [
2,3,4], [1,2,3]) # Four operands Set:[2, 3] >> x.union(r
ange(10)) # Non-MultiSets work, too Set:[1, 2, 3, 4, 0,
5, 6, 7, 8, 9] >> w = MultiSet('soap') # String sets >>
w Set(['s','o','a','p']) >> ''.join(w |'super')'soapuer'>> (
w |'super') & MultiSet('slots') Set(['s','o'])
```

中文翻译

5. 集合对象：下面是你会得到的交互结果。要让 Chapter3 2/setwrapper.py 的导入生效，要么在它所在的文件夹里运行，要么把它复制到你的工作目录，要么按第五部分的方法把它的文件夹加进导入搜索路径。注释解释了调用了哪些方法。同时记住：集合是 Python 的内置类型，所以这基本只是个编码练习（集合更多见第 5 章）：

```
>>> from setwrapper import Set >>> x = Set([1, 2, 3, 4]) # 调用 init
(构造时去重) >>> y = Set([3, 4, 5]) >>> x & y # 调 a
nd → intersect → 打印时调 repr Set:[3, 4] >>> x | y #
```

调 or → union → repr Set:[1, 2, 3, 4, 5] >>> z = Set('hello') # init 去重 → {h,e,l,o} >>> z[0], z[-1], z[2:] # 调 getitem ('h','o', ['l','o']) >>> for c in z: print(c, end='') # 调 iter (否则退回 getitem 协议) h e l o >>> ''.join(c.upper() for c in z) # 同样走 iter'HELO'>>> len(z), z # len、repr (4, Set:['h','e','l','o']) >>> z & 'mello', z | 'mello' # 字符串也能参与运算(Set:['e','l','o'], Set:['h','e','l','o','m']) 多操作数扩展子类的一种方案看起来像示例 B-26 中的类。它只需要替换原集中的两个方法。类的文档字符串说明了它的工作原理：python from setwrapper import Set class MultiSet(Set): """继承 Set 的所有名字，但把 intersect 和 union 扩展为支持多个操作数。注意 "self" 仍是第一个参数（现在被装进 args 参数里）。还要注意，继承的 & 和 | 运算符会用 2 个参数调用这里的新方法，但处理 2 个以上操作数需要方法调用，而不是表达式。intersect 在这里不去重：由 Set 构造器负责去重。""" def intersect(self, others): res = [] for x in self: # 扫描第一个序列 for other in others: # 对所有其他参数 if x not in other: break # x 是否在每一个里？ else: # 不在：跳出循环 res.append(x) # 在：把元素加入结果 return Set(res) def union(args): # self 就是 args[0] res = [] for seq in args: # 遍历所有参数 for x in seq: # 遍历所有元素 if not x in res: res.append(x) # 新元素才加入结果 return Set(res) 与这个扩展的交互大致如下。注意：你可以用 & 或调用 intersect 做交集，但三个及以上操作数必须调用 intersect；& 是二元（两侧）运算符。另外，如果把原类以 setwrapper.Set 的方式引用（import 的 as 子句也可以按需给类改名），我们本来可以把 MultiSet 直接叫做 Set，让这个改动更透明： >>> x

```
& y, x | y # 两个操作数(Set:[3, 4], Set:[1, 2, 3, 4, 5]) >>
> x.intersect(y, z) # 三个操作数Set:[] >>> x.union(y, z)
Set:[1, 2, 3, 4, 5, 0] >>> x.intersect([1,2,3], [2,3,4], [1,2
,3]) # 四个操作数Set:[2, 3] >>> x.union(range(10)) # 非
MultiSet 也适用Set:[1, 2, 3, 4, 0, 5, 6, 7, 8, 9] >>> ''.join
(w |'super') # 字符串集合'soapuer'>>> (w |'super') &
MultiSet('slots') Set(['s','o'])
```

代码分析（逐行解读）

```
class MultiSet(Set):
    def intersect(self, *others):
        res = []
        for x in self:
            for other in others:
                if x not in other: break
            else:
                res.append(x)
        return Set(res)

    def union(*args):
        res = []
        for seq in args:
            for x in seq:
                if not x in res:
                    res.append(x)
        return Set(res)
```

· $x \& y$ 的求值链: $x.\text{and}(y) \rightarrow$ (继承 Set 的 and) $\rightarrow \text{self}.\text{intersect}(y) \rightarrow \text{MultiSet}.\text{intersect}$ 接收 (self, y)。

· 三个操作数 $x.\text{intersect}(y, z)$: $\&$ 是二元运算符, 语法上无法写 $x \& y \& z$ 之外多参数, 所以多操作数必须方法调用。

深度理解

- 核心概念：继承 + 方法覆写让"集合"支持任意多个操作数的交集/并集；for...else（无 break 走 else）是实现"全通过"语义的精妙结构。
- self 也是 args 的一员：union(args) 里 self 就是 args[0]——方法调用 x.union(y,z) 等价 union(x, y, z)。
- &| 是二元运算符：最多两个操作数；多操作数必须调用方法——这是运算符重载的天然边界。
- 通用性：union(range(10))、'mello'、'super' 都能参与——因为实现只要求操作数是可迭代对象（x not in other 做的是迭代式成员测试）。
- 为何构造时去重：Set(res) 构造器内部会去掉重复元素，所以 intersect/union 只管"收集"。
- 常见误区：把 x in y 写成 y in x；忘记 for...else 的语义（有 break 就不走 else）；以为 & 能带 3 个操作数。

题 6. Class tree links (类树链接)

英文原文

6. Class tree links: Example B-27 lists one way to change the lister class in Example 31-10, along with a rerun of the associated tester to show its augmented format. For full credit, do the same for the dir-based version, and also do this when formatting class objects in the tree-climber variant.

To import testmixin.py as a test, either copy it over from the Chapter 31 examples folder or add that folder to sys.path as we did earlier in Part IV's solutions. It was copied here for variety: Example B-27. Part6/listinstance-mod.py

```
python class ListInstance: def attrnames(self):...unchanged...
```

```
def str(self): return (f'<Instance of {self.class.name}' #  
My class's name f'({self.supers()}),' # My class's supers  
f'address {id(self):#x}: ' # My address (hex) f'{self.attrn  
ames()}>') # name=value list
```

```
def supers(self): names = [] for super in self.class.bas  
es: # One level up from class names.append(super.na  
me) # name, not str(super)
```

```
return',' .join(names) # Or:',' .join(super.name for super  
in self.class.bases) if name == 'main': import testmixin  
# Assume testmixin.py copied to "." testmixin.testter(L  
istInstance) # Test class in this module
```

```
$ python3 listinstance-mod.py <Instance of Sub(Sup  
er, ListInstance), address 0x10edc66c0:
```

```
data1='code'data2='Python'data3=3.12 >
```

中文翻译

6. 类树链接：示例 B-27 列出了修改示例 31-10 中 lister 类的一种方式，并重跑关联的测试器以展示增强后的格式。

要得满分，请对基于 `dir` 的版本做同样处理，并且在对类对象格式化时也要在树形攀登（`tree-climber`）变体中这样做。要导入 `testmixin.py` 作为测试，要么从第 31 章的示例文件夹复制它，要么像我们之前在第四部分解答里那样把那个文件夹加进 `sys.path`。这里为多样化起见把它复制了过来：

```
python class ListInstance: # 混入类 (mixin) : 给任何类附加打印能力
def attrnames(self):...保持不变...
def str(self): return (f'<Instance of {self.class.name}' # 我的类名
f'({self.supers()}),' # 我的父类们
f'address {id(self):#x}:' # 我的地址 (十六进制)
f'{self.attrnames()}>') # 属性名=值列表
def supers(self): names = []
for super in self.class.bases: # 从类往上走一级
names.append(super.name) # 用名字，而不是 str(super)
return ','.join(names) # 或: ','.join(super.name for super in self.class.bases)
if name == 'main': import testmixin # 假设 testmixin.py 已复制到 "."
testmixin.testter(ListInstance) # 测试本模块里的类
$ python3 listinstance-mod.py <Instance of Sub(Super, ListInstance), address 0x10edc66c0: data1='code'data2='Python'data3=3.12 >
```

代码分析（逐行解读）

```
class ListInstance:                                # mixin ...
    def __str__(self):                              # print()
        return (f'<Instance of {self.__class__.__name__}' # self...
                f'({self.__supers()}),' # ...
                f'address {id(self):#x}:' # id(s...
                f'{self.__attrnames()}>') # ...
    def __supers(self):
        names = []
        for super in self.__class__.__bases__: # __bases__...
```

```
names.append(super.__name__)      #
return ', '.join(names)          #
```

- `self.class` 指向实例所属的类对象；`bases` 是该类直接父类的元组（"类树"的一级链接）。
- `id(self)` 返回对象的内存地址（CPython 中），`:#x` 格式化成 `0x10edc66c0` 样式。
- 输出 `<Instance of Sub(Super, ListInstance), address 0x10edc66c0: ...>`：显示了类名、父类名（类树链接）和属性。

深度理解

- 核心概念：类树（class tree）是 Python OOP 的组织结构——类通过 `bases` 链接到父类，实例通过 `class` 链接到类。属性查找就是沿这条树向上搜索（MRO）。
- 为什么这样实现：str 混入类（mixin）让任何多继承它的类自动获得友好的打印输出——这是"用组合代替复制"的横切关注点（cross-cutting concern）模式。
- `bases` 只给直接父类：向上多级需要循环或递归遍历 `super.bases`——树形攀登（tree-climber）变体就是在做这件事。
- 常见误区：把 `super`（内置函数）当变量名用会遮蔽内置；`str(super)` 是 `<class ...>` 表示，想要名字必须用 `super.name`；`str` 返回字符串而非打印。

题 7. Composition (组合: 午餐模拟)

英文原文

7. Composition: A full-points solution is coded in Example B-28, with comments from the description mixed in with the code. This is one case where it's probably easier to express a problem in code than it is in narrative: Example B-28. Part6/lunch.py

```
python class Lunch: def init(self): # Make/embed Customer, Employee self.cust = Customer() self.empl = Employee() def order(self, foodName): # Start Customer order simulation self.cust.placeOrder(foodName, self.empl) def result(self): # Ask the Customer about its Food self.cust.printFood() class Customer: def init(self): # Initialize my food to None self.food = None def placeOrder(self, foodName, employee): # Place order with Employee self.food = employee.takeOrder(foodName) def printFood(self): # Print the name of my food print(self.food.name) class Employee: def takeOrder(self, foodName): # Return Food, with desired name return Food(foodName) class Food: def init(self, name): # Store food name self.name = name if name == 'main': x = Lunch() # Self-test code x.order('burritos') # If run, not imported x.result() x.order('pizza') x.result()
```

When run, customers place orders and get food from employees. This could be much more involved, but

it suffices to demo the routing of messages between objects that's typical in OOP code:

```
$ python3 lunch.py burritos pizza
```

中文翻译

7. 组合：一个满分解法编码在示例 B-28 中，题目描述中的注释与代码混排在一起。这大概是一个"用代码表达问题比用文字叙述更容易"的例子：

```
python class Lunch: # 组合器：把 Customer 和 Employee 装配起来
def init(self): # 创建/嵌入 Customer、Employee
self.cust = Customer()
self.empl = Employee()
def order(self, foodName): # 启动 Customer 的订餐模拟
self.cust.placeOrder(foodName, self.empl)
def result(self): # 询问 Customer 关于它的 Food
self.cust.printFood()
class Customer: # 顾客
def init(self): # 初始化我的食物为 None
self.food = None
def placeOrder(self, foodName, employee): # 向 Employee 下订单
self.food = employee.takeOrder(foodName)
# 拿到 Food 对象存入自己
def printFood(self): # 打印我的食物名字
print(self.food.name)
class Employee: # 员工
def takeOrder(self, foodName): # 返回带所需名字的 Food
return Food(foodName)
class Food: # 食物
def init(self, name): # 存食物名
self.name = name
if name == 'main': # 自测代码
x = Lunch() # 运行而不是导入时才执行
x.order('burritos')
x.result()
x.order('pizza')
x.result()
$ python3 lunch.py burritos pizza
```

代码分析（逐行解读）

```
class Lunch:                                # ""Lunch has-aCustomer ...
    def __init__(self):
        self.cust = Customer() #
        self.empl = Employee()
    def order(self, foodName):
        self.cust.placeOrder(foodName, self.empl) # Lunch ""...
    def result(self):
        self.cust.printFood()

class Customer:
    def placeOrder(self, foodName, employee):
        self.food = employee.takeOrder(foodName) # ...
    def printFood(self):
        print(self.food.name) #
```

消息路由链：x.order('burritos') → Customer.placeOrder → Employee.takeOrder → 创建 Food('burritos') → 存进 customer.food → x.result() → printFood → 打印 burritos。

深度理解

·核心概念：组合（composition）——一个对象“拥有”其他对象（has-a 关系），通过消息传递（message passing）协作：Lunch 调度、Customer 下单、Employee 制作、Food 描述。这是 OOP 代码中典型的“对象间路由”。

·与继承的区别：组合强调“部分-整体”装配（Lunch 有 Customer），继承强调“是”关系（Cat 是 Mammal）。优先组合、必要时继承是公认的设计原则。

- 职责划分：每个类只做一件事——Customer 存食物、Employee 造食物、Food 存名字；类间通过公开方法协作，内部细节不外泄（封装）。
 - 运行入口：if name == 'main': 保证作为模块导入时静默，作为脚本运行时执行演示。
 - 常见误区：把组合写成“深层链式调用”（如 x.cust.food.name）破坏封装——通过方法暴露才是正道；忘了给类设计“协作接口”导致对象无法互相调用。
-

题 8. Zoo animal hierarchy（动物园动物层次）

英文原文

8. Zoo animal hierarchy: Example B-29 shows one way to code the taxonomy in Python; it's artificial, but the general coding pattern applies to many real structures, from GUIs to employee databases to spacecraft. Notice that the self.speak call in Animal triggers an independent inheritance search, which generally finds speak in a subclass. Test this interactively by calling the reply method for instances per the exercise description. Try extending this hierarchy with new classes and making instances of various classes in the tree: Example B-29. Part6/zoo.py

```
python class Animal:
    def reply(self):
        self.speak() # Back to subclass
    def speak(self):
        print('blah') # Custom message
class Mammal(Animal):
```

```

): def speak(self): print('huh?') class Cat(Mammal): def
speak(self): print('meow') class Dog(Mammal): def sp
eak(self): print('bark') class Primate(Mammal): def spe
ak(self): print('Hello world!') class Hacker(Primate): pa
ss # Inherit from Primate

```

中文翻译

8. 动物园动物层次：示例 B-29 展示了在 Python 中编写这个分类体系的一种方式；它是人为构造的，但这种通用编码模式适用于许多真实结构——从 GUI 到员工数据库再到航天器。注意：Animal 里的 self.speak 调用会触发一次独立的继承搜索，通常会在子类里找到 speak。按练习描述，通过为实例调用 reply 方法来交互式测试它。试着用新类扩展这个层次结构，并创建树中各种类的实例：python

```

class A
nimal: # 根类：定义通用协议def reply(self): self.speak()
# 转发给 self.speak ——动态分派def speak(self): print('
blah')# 默认实现 ("自定义消息") class Mammal(Anima
l): # 一级子类def speak(self): print('huh?') class Cat(Ma
mmal): def speak(self): print('meow') class Dog(Mam
mal): def speak(self): print('bark') class Primate(Mam
mal): def speak(self): print('Hello world!') class Hacker
(Primate): pass # 空类体：全部继承自 Primate

```

代码分析（逐行解读）

```

class Animal:
    def reply(self): self.speak() # self.speak—
    def speak(self): print('blah') #

class Mammal(Animal):
    def speak(self): print('huh?')

class Cat(Mammal):
    def speak(self): print('meow')

class Hacker(Primate): pass # passspeak Primate

```

交互测试（预期行为）：

```

>>> cat = Cat(); dog = Dog(); hacker = Hacker()
>>> cat.reply() # → meowreply self.speak() Cat→Mammal→Anim...
>>> dog.reply() # → bark
>>> hacker.reply() # → Hello world!Hacker speak Primate

```

深度理解

- 核心概念：多态分派（dynamic dispatch）——Animal.reply 中的 self.speak() 不是"调用 Animal.speak"，而是"以实例的类型为起点、沿继承树向上查找 speak"。因此同一句 reply() 在不同实例上发出不同的叫声。
- 底层机制：self.speak 是属性查找：实例 → 实例的类 → 父类 →（MRO）。每新建一层 speak 覆写，分派结果就改变——这就是"覆写（override）"的威力。
- 空类的意义：class Hacker(Primate): pass 表示"不做任何改动、完全继承"——用于给分类体系占位或将来添加特殊行为。
- 设计模式对应：这本质是"模板方法"的又一例（骨架 reply 固定，可变点 speak 由子类决定），也常被称为"策略

的继承实现”。

·常见误区：在 reply 里写 `Animal.speak(self)` 会绕过多态（永远用基类实现）；忘记覆写导致所有动物都喊 "blah"。

Part VII: Exceptions (异常)

英文原文

Part VII, Exceptions See "Test Your Knowledge: Part V II Exercises" in Chapter 36 for the exercises.

中文翻译

第七部分：异常。练习见第 36 章的 "Test Your Knowledge: Part VII Exercises"。

深度理解

·核心概念：Part VII 练习覆盖 try/except/else 结构、自定义异常类、异常的"冒泡 (percolation)"、`sys.excinfo`、`traceback` 打印、装饰器包装异常，以及 10 个自学演示程序 (Self-Study Demos)。

·练习主线：从一个会抛异常的 `oops` 函数出发，逐步把它改造成"能捕获、能识别类型、能记录 `traceback`"的健壮异常处理工具。

题 1. try / except

英文原文

1. try/except: One possible coding of the oops function is listed in Example B-30. As for the noncoding questions, changing oops to raise a `KeyError` instead of an `IndexError` means that the try handler won't catch the exception—it "percolates" to the top level and triggers Python's default error message. The names `KeyError` and `IndexError` come from the outermost built-in names scope (the B in "LEGB"). Import builtins and pass it as an argument to the `dir` function to see this for yourself, per Chapter 17: Example B-30.

```
Part7/oops.py python def oops(): raise IndexError() def doomed(): try: oops() except IndexError: print('caught an index error!') else: print('no error caught...') if name == 'main': doomed() $ python3 oops.py caught an index error!
```

中文翻译

1. try/except: oops 函数的一种可行写法列在示例 B-30 中。至于非编码问题：把 oops 改为抛出 `KeyError` 而不是 `IndexError`，意味着 try 处理器不会捕获该异常——它会“冒泡 (percolates) ”到顶层，触发 Python 的默认错误消息。`KeyError` 和 `IndexError` 这两个名字来自最外层的内置名字作用域 (LEGB 中的 B)。按第 17 章的方法，导入 `builtins` 并把它作为参数传给 `dir` 函数，你自己就能看到这一

点: python

```
def oops(): raise IndexError() # 抛出一个 IndexError 实例
def doomed(): try: oops() # 调用可能抛异常的函数
except IndexError: # 只捕获 IndexError 类型
    print('caught an index error!')
else: # 没有异常时才执行
    print('no error caught...')
if name == 'main': doomed()
$ python3 oops.py
caught an index error!
```

代码分析（逐行解读）

```
def oops():
    raise IndexError()          # raise

def doomed():
    try:                        # try
        oops()
    except IndexError:         #
        print('caught an index error!')
    else:                      # except
        print('no error caught...')

if __name__ == '__main__': doomed()
```

深度理解

- 核心概念：try/except/else 是异常处理的骨架——try 监控、except 捕获指定类型、else 在"无异常"时执行（区别于 finally 的无条件执行）。
- 异常匹配：except IndexError 用类匹配，不只精确类型——IndexError 的子类也会被捕获。改成 KeyError 后，except IndexError 不再匹配，异常沿调用栈一路冒泡到顶层，由默认处理器打印 Traceback。
- 名字来源（LEGB）：KeyError、IndexError 是内建命名

空间 (builtins, 即 LEGB 的 B) 里的名字——import builtins; dir(builtins) 就能看到它们。异常类型本身就是"继承自 Exception 的类对象"。

·设计思想：异常让"错误处理"与"业务逻辑"分离——调用方法决定如何应对，而不是每个函数都自己打印错误。

·常见误区：捕获"裸 except"（捕获一切，包括 KeyboardInterrupt）是不良实践；else 与 finally 语义混淆。

题 2. Exception objects and lists (异常对象与异常列表)

英文原文

2. Exception objects and lists: Example B-31 is one way to extend this module for an exception of its own: Example B-31. Part7/oops2.py

```
python class MyError(Exception): pass def oops(): raise MyError('Hack!') def doomed(): try: oops()
```

```
except IndexError: print('caught an index error!') except MyError as exc: print('caught error:', MyError, exc) else: print('no error caught...') if name == 'main': doomed()
```

```
$ python3 oops2.py caught error: <class'main.MyError'> Hack!
```

Like all class exceptions, the raised instance is accessible via the as variable data; the error message shows

both the class's (<...>) and its instance's (Hack!) displays. The instance must be inheriting both an `__init__` and a `__repr__` or `__str__` from Python's `Exception` class, or it would print much as the class does. See Chapter 35 for details on how these defaults work in built-in exception classes.

中文翻译

2. 异常对象与异常列表：示例 B-31 是把该模块扩展出自定义异常的一种方式：

```
python class MyError(Exception):
    pass # 自定义异常：继承内置的 Exception 基类
def oops():
    raise MyError('Hack!') # 抛出自定义异常，带一条消息参数
def doomed():
    try:
        oops()
    except IndexError: # 第一个 except：只匹配 IndexError
        print('caught an index error!')
    except MyError as exc: # 第二个 except：匹配 MyError，用 as 绑定异常实例
        print('caught error:', MyError, exc)
    else:
        print('no error caught...')
if name == 'main':
    doomed()
$ python3 oops2.py
caught error: <class 'main.MyError'> Hack!
```

与所有类异常一样，被抛出的实例可以通过 `as` 变量访问；错误消息同时显示类的表示 (<...>) 和实例的表示 (Hack!)。该实例必须从 Python 的 `Exception` 类同时继承 `__init__` 和 `__repr__` 或 `__str__`，否则它打印起来会跟类本身差不多。关于这些默认值在内置异常类中如何工作，详见第 35 章。

代码分析（逐行解读）

```

class MyError(Exception): pass          # = Exception
                                         # __init__(args).__str__/_...

def oops():
    raise MyError('Hack!')              # MyError('Hack!') 'Hack!...'

def doomed():
    try:
        oops()
    except IndexError:                   # IndexError
        print('caught an index error!')
    except MyError as exc:               # MyErrorexc
        print('caught error:', MyError, exc)
                                         # MyError <class '__main__...
                                         # exc 'Hack!' __str__ ...
    else:
        print('no error caught...')

```

深度理解

- 核心概念：异常是类，raise 抛的是实例。except Some Error as e: 里的 e 就是那个实例，可以读取、检查、重新抛出。
- 自定义异常的最小写法：class MyError(Exception): pass 就够了——Exception 基类提供了默认的 init（把参数存进 args）和 str/repr（打印 args）。MyError('Hack!') 的'Hack!' 就是一条"错误消息"。
- as 绑定的作用域：Python 3 中 as 变量在 except 块结束后即被清除（防止悬垂引用）。
- 多个 except 的顺序：从上到下逐一匹配，命中第一个即停——所以子类必须写在父类之前（先 except MyError 再 except Exception，反过来就永远轮不到 MyError）

- 常见误区：except MyError as exc 里打印 MyError 得到的是类对象而不是消息；忘了自定义异常继承 Exception（继承普通类也行，但 Exception 保证兼容性）。

题 3. Error handling（错误处理工具）

英文原文

3. Error handling: Example B-32 is one way to solve this exercise. It codes tests in a file rather than interactively, but the results are similar enough for full credit. Notice that the empty except and sys.excinfo approach used here will catch exit-related exceptions that listing Exception with an as variable won't; that's probably not ideal in most applications code but might be useful in a tool like this designed to work as a sort of exceptions firewall.

Example B-32. Part7/exctools.py

```
python import sys, traceback def safe(callee, pargs, kargs): try: callee(pargs, kargs) # Catch everything else except: # Or "except Exception as E:" traceback.printexc()
```

```
print(f'Got {sys.excinfo()[0]} {sys.excinfo()[1]}') if name == 'main': import oops2 safe(oops2.oops)
```

```
$ python3 exctools.py Traceback (most recent call la
```

st): File ".../LP6E/AppendixB/Part7/exctools.py", line 5, in safe callee(pargs, kargs) # Catch everything else
File ".../LP6E/AppendixB/Part7/oops2.py", line 4, in oops

```
raise MyError('Hack!') oops2.MyError: Hack! Got <class 'oops2.MyError'> Hack!
```

Bonus points: the sort of code in Example B-33 could turn this into a function decorator that could wrap and catch exceptions raised by any function, using techniques introduced briefly in Chapter 32, but covered more fully in Chapter 39—it augments a function, rather than expecting it to be passed in explicitly, and produces similar output when run (there's an extra call level, and filenames differ): Example B-33. Part7/exctoolsdeco.py

```
python import sys, traceback def safe(callee): def callproxy(pargs, kargs): try: return callee(pargs, kargs) except Exception as E: traceback.printexc()
```

```
print(f'Got {E.class} {E}') return callproxy if name == 'main': import oops2 @safe def test(): # test = safe(test) oops2.oops() test()
```

中文翻译

3. 错误处理：示例 B-32 是解决这个练习的一种方式。它把测试写在文件里而不是交互式地运行，但结果足够相似、可得满分。注意：这里使用的空 `except` 和 `sys.excinfo` 方案

会捕获与退出 (exit) 相关的异常，而 Exception 加 as 变量的写法捕获不到；这在大多数应用代码中可能并不理想，但对这种设计成“异常防火墙”的工具可能有用：

```
python import sys, traceback
def safe(callee, pargs, kargs): # 通用安全调用器
try: callee(pargs, kargs) # 调用任意函数 (位置+关键字参数)
except: # 空 except: 捕获一切 (包括 SystemExit 等)
traceback.printexc() # 打印完整回溯
print(f'Got {sys.excinfo()[0]} {sys.excinfo()[1]}') # sys.excinfo() 返回 (类型, 实例, 回溯对象)
if name == 'main':
import oops2
safe(oops2.oops) $ python3 exctools.py
Traceback (most recent call last):
File ".../exctools.py", line 5, in safe
callee(pargs, kargs) # 抛出点
File ".../oops2.py", line 4, in oops
raise MyError('Hack!')
oops2.MyError: Hack!
Got <class 'oops2.MyError'> Hack!
加分项：
示例 B-33 中的代码可以把这变成一个函数装饰器，包装并捕获任何函数抛出的异常——使用第 32 章简要介绍、第 39 章更全面讲解的技术；它增强 (augments) 一个函数，而不是期待函数被显式传入，运行输出也类似 (多一层调用，文件名不同)：
```

```
python import sys, traceback
def safe(callee): # 装饰器工厂：接收函数
def callproxy(pargs, kargs): # 代理函数：原函数的“替身”
try: return callee(pargs, kargs)
except Exception as E: # 捕获 Exception 及其子类
traceback.printexc()
print(f'Got {E.class} {E}')
return callproxy # 返回代理，替换原函数名
if name == 'main':
import oops2
@safe # 语法糖：test = safe(test)
def test(): oops2.oops()
test()
```

代码分析（逐行解读）

```
def safe(callee, *pargs, **kargs):    # """pargs ...
    try:
        callee(*pargs, **kargs)        #
    except:                             # except =
        traceback.print_exc()          # print_exc traceback...
        print(f'Got {sys.exc_info()[0]} {sys.exc_info()[1]}')
                                          # sys.exc_info() → (, ...
```

装饰器版关键点：

```
def safe(callee):
    def callproxy(*pargs, **kargs):    # callee
        try:
            return callee(*pargs, **kargs)    #
        except Exception as E:            # Exception Sys...
            traceback.print_exc()
            print(f'Got {E.__class__} {E}')
    return callproxy                    # ""

@safe
def test(): ...    # test = safe(test)—
```

深度理解

- 核心概念：异常"防火墙"模式——把任意函数包在 try 里，异常发生时既打印回溯又不让程序崩溃。两种包装方式：显式传函数（safe(f)）与装饰器（@safe）。
- 空 except vs except Exception：空 except: 捕获包括 SystemExit、KeyboardInterrupt 在内的一切（exit-related）；except Exception as E: 只捕获普通异常子树。工具型"防火墙"用前者（宁可吞掉一切），应用代码用后者更安全。

- `sys.excinfo()`：返回当前异常的三元组 (类型, 实例, traceback 对象)——[0] 是类 (`<class'oops2.MyError'>`)、[1] 是实例 (打印出 Hack!)。它在 3.x 中比 Python 2 的裸 `sys.exc_type/sys.exc_value` 更规范。
- 装饰器的机制：@safe 在函数定义完成时执行 `safe(test)`，用 `callproxy` 替换名字 `test`；此后调用 `test()` 实际调用代理——闭包捕获原函数，“增强而非修改”。
- 闭包与 args/kwars：代理用 `pargs, kargs` 原样转发任意签名，保证接口透明——这是所有装饰器/包装器的标配。
- 常见误区：裸 `except` 吞掉 `Ctrl+C` (`KeyboardInterrupt`)；装饰器忘了 `return` (函数变 `None`)；`sys.excinfo` 在 `except` 块外调用返回 (`None, None, None`)。

题 4. Self-study examples (自学演示程序)

英文原文

4. Self-study examples: In closing, Examples B-34 through B-43 are 10 examples for you to study on your own. Their code and supporting files are in the Self-Study-Demos subfolder of the examples package's AppendixB/Part7 folder. These require no extra installs as they use standard-library tools, though tkinter is sketchy on phones (see Appendix A). For more examples, see follow-up books and resources for the application domains you'll be exploring next:

中文翻译

4. 自学演示程序：最后，示例 B-34 到 B-43 是 10 个供你自行研究的例子。它们的代码和支持文件在示例包 AppendixB/Part7 文件夹的 Self-Study-Demos 子文件夹中。它们只使用标准库工具，无需额外安装，不过 tkinter 在手机上不太靠谱（见附录 A）。想要更多例子，请看后续书籍和相关应用领域的资源：

示例 B-34 到 B-36：找出最大的 Python 源文件（三个版本）

```
# Example B-34. Part7/Self-Study-Demos/largest-dir.py
# Python
import os, glob
dirname = '/Users/me/Downloads'          # input() s...
allsizes = []
allpy = glob.glob(dirname + os.sep + '*.py')  # glob .py ...
for filename in allpy:
    filesize = os.path.getsize(filename)
    allsizes.append((filesize, filename))      # (, )

allsizes.sort()                             #
print(allsizes[:2])                         #
print(allsizes[-2:])                       #
```

```
# Example B-35. Part7/Self-Study-Demos/largest-tree.py
# Python
import sys, os, pprint
if sys.platform[:3] == 'win':
    dirname = r'C:\Users\me\Downloads'      # Windows raw \
else:
    dirname = '/Users/me/Downloads'
allsizes = []
for (thisDir, subsHere, filesHere) in os.walk(dirname):  # os.wal...
```

```

    for filename in filesHere:
        if filename.endswith('.py'):
            fullname = os.path.join(thisDir, filename)      #
            fullsize = os.path.getsize(fullname)
            allsizes.append((fullsize, fullname))
    allsizes.sort()
    pprint.pprint(allsizes[:2])                          # pprint
    pprint.pprint(allsizes[-2:])

```

```

# Example B-36. Part7/Self-Study-Demos/largest-import.py
# Python
import sys, os, pprint
visited = {}                                             #
allsizes = []
for srcdir in sys.path:                                #
    for (thisDir, subsHere, filesHere) in os.walk(srcdir):
        thisDir = os.path.normpath(thisDir)            # ...
        if thisDir.upper() in visited:                  # Window...
            continue
        else:
            visited[thisDir.upper()] = True
        for filename in filesHere:
            if filename.endswith('.py'):
                pypath = os.path.join(thisDir, filename)
                try:
                    pysize = os.path.getsize(pypath)
                except:                                   #
                    print('skipping', pypath)
                allsizes.append((pysize, pypath))
    allsizes.sort()
    pprint.pprint(allsizes[:3])
    pprint.pprint(allsizes[-3:])

```

深度理解：

·三个版本分别演示 glob（单目录通配）、os.walk（递归树遍历）、sys.path 遍历（import 搜索路径）+ visited

去重——同一任务的三种"范围"。

· (size, name) 元组 + sort(): 利用元组比较"先比第一个元素"的特性, 天然按大小排序; pprint.pprint 适合打印大结构。

· 路径处理: os.sep (平台分隔符)、os.path.join (安全拼路径)、os.path.normpath (规范化)、raw 字符串防 \ 转义。

示例 B-37 与 B-38: 按列求和 (summer)

```
# Example B-37. Part7/Self-Study-Demos/summer1.py
#
filename = 'data.txt'
sums = {}
for line in open(filename):
    cols = line.split(',')
    nums = [int(col) for col in cols]
    for (ix, num) in enumerate(nums):
        sums[ix] = sums.get(ix, 0) + num # dict.get(key, 0)...
for key in sorted(sums):
    print(key, '=', sums[key])
```

```
# Example B-38. Part7/Self-Study-Demos/summer2.py
# summer1
import sys
filename = sys.argv[1]
numcols = int(sys.argv[2])

totals = [0] * numcols
for line in open(filename):
    cols = line.split(',')
    nums = [int(x) for x in cols]
    totals = [(x + y) for (x, y) in zip(totals, nums)]
print(totals)
```

深度理解：

- summer1：字典版——get(key, 0) 处理"列首次出现"； enumerate 提供列号。列数未知也能跑。
- summer2：列表版——列数必须预先知道（命令行传入）； zip(totals, nums) 逐对相加再生成新列表。
- 对比两种数据结构：字典灵活（键任意）、列表高效（下标直取）——"选型即设计"。

示例 B-39：回归测试 (regrtest)

```
# Example B-39. Part7/Self-Study-Demos/regrtest.py
#
import os
testscripts = [dict(script='test1.py', args=''), # glo...
               dict(script='test2.py', args='-opt')] #
for testcase in testscripts:
    commandline = '%(script)s %(args)s' % testcase # % ...
    output = os.popen(commandline).read() # ...
    result = testcase['script'] + '.result' #
    if not os.path.exists(result):
        open(result, 'w').write(output) #
        print('Created:', result)
    else:
        priorresult = open(result).read() #
        if output != priorresult:
            print('FAILED:', testcase['script']) # → ...
            print(output)
        else:
            print('Passed:', testcase['script'])
```

深度理解：

- 回归测试 (regression test) 思想：把程序首次输出存为"基线 (baseline)", 以后每次运行对比——输出变了就

报 FAILED。这是软件工程里自动化测试的雏形。

· `os.popen` 运行子命令并读取其输出（在 3.x 中可用 `subprocess` 更规范）；测试脚本列表用字典列表描述"脚本名 + 参数"。

示例 B-40 与 B-41: tkinter GUI (两个版本)

```
# Example B-40. Part7/Self-Study-Demos/gui1.py
"""
tkinter GUI
GUI
"""
from tkinter import *
import random
fontsize = 25
colors = ['red', 'green', 'blue', 'yellow', 'orange', 'white', 'cy...

def reply(text):
    print(text)
    popup = Toplevel() #
    color = random.choice(colors) #
    Label(popup, text='Popup', bg='black', fg=color).pack()
    L.config(fg=color) #

def cycle():
    L.config(fg=random.choice(colors))
    win.after(250, cycle) # 250ms cycle...

def grow():
    global fontsize # global
    fontsize += 5
    L.config(font=('arial', fontsize, 'italic'))
    win.after(100, grow) # 100ms

win = Tk() #
L = Label(win, text='Hack',
```

```

        font=('arial', fontsize, 'italic'), fg='yellow', bg='nav...
        relief=RAISED)
L.pack(side=TOP, expand=YES, fill=BOTH)
Button(win, text='popup', command=(lambda: reply('new'))).pack(sid...
Button(win, text='cycle', command=cycle).pack(side=BOTTOM, fill=X)
Button(win, text='grow', command=grow).pack(side=BOTTOM, fill=X)
win.mainloop()                                # GUI

```

```

# Example B-41. Part7/Self-Study-Demos/gui2.py
"""
    gui1
    GUI  Stop
"""
from tkinter import *
import random

class MyGui:
    """
        GUI
    """
    colors = ['blue', 'green', 'orange', 'red', 'brown', 'yellow']
    def __init__(self, parent, title='popup'):
        parent.title(title)
        self.growing = False                #
        self.fontsize = 10
        self.lab = Label(parent, text='Hack2', fg='white', bg='nav...
        self.lab.pack(expand=YES, fill=BOTH)
        Button(parent, text='Hack', command=self.reply).pack(side=...
        Button(parent, text='Grow', command=self.grow).pack(side=L...
        Button(parent, text='Stop', command=self.stop).pack(side=L...
    def reply(self):
        " Hack "
        self.fontsize += 5
        color = random.choice(self.colors)
        self.lab.config(bg=color,
                        font=('courier', self.fontsize, 'bold ital...
    def grow(self):
        " Grow "

```

```

        self.growing = True
        self.grower()
    def grower(self):
        """
        if self.growing:                                # Stop
            self.fontsize += 5
            self.lab.config(font=('courier', self.fontsize, 'bold'...)
            self.lab.after(500, self.grower) # 500ms
    def stop(self):
        " Stop "
        self.growing = False

class MySubGui(MyGui):
    colors = ['black', 'purple']                        #

MyGui(Tk(), 'main')                                    #
MyGui(Toplevel())
MySubGui(Toplevel())
mainloop()

```

深度理解：

- tkinter 事件驱动模型：mainloop() 进入事件循环；after(毫秒, 函数) 是"定时器回调"——GUI 的动画/周期任务都靠它，而不是死循环（会卡死界面）。
- gui1（过程式）vs gui2（类）：gui2 把状态（字体大小、生长开关）收进实例属性，多个窗口互不干扰，还支持子类覆写（MySubGui 换颜色）——类把 GUI 组件和状态打包成可复用单元，这就是 Part VI 学的 OOP 在真实 GUI 里的应用。
- global fontsize：函数内修改模块级变量必须先声明 global，否则会创建局部变量。

· command=... 绑定回调; lambda 用于给回调传参。

示例 B-42: POP 邮件检查工具 (popmail)

```
# Example B-42. Part7/Self-Study-Demos/popmail.py
"""
POP
pop
"""

import poplib, getpass, sys

mailserver = 'your pop email server name here' # pop.server...
mailuser = 'your pop email user name here' # userid
mailpasswd = getpass.getpass(f'Password for {mailserver}? ') # ...

print('Connecting...')
server = poplib.POP3(mailserver) # POP3
server.user(mailuser) #
server.pass_(mailpasswd) #
try:
    print(server.getwelcome()) #
    msgCount, mboxSize = server.stat() #
    print('There are', msgCount, 'mail messages, size ', mboxSize)
    msginfo = server.list() #
    print(msginfo)
    for i in range(msgCount):
        msgnum = i+1
        msgsize = msginfo[1][i].split()[1] #
        resp, hdrlines, octets = server.top(msgnum, 0) # ...
        print('-'*80)
        print('%d: octets=%d, size=%s' % (msgnum, octets, msgsiz...
        for line in hdrlines: print(line)
        if input('Print?') in ['y', 'Y']:
            for line in server.retr(msgnum)[1]: print(line) # ...
        if input('Delete?') in ['y', 'Y']:
            print('deleting')
            server.dele(msgnum) #
    else:
```

```

        print('skipping')
finally:
    server.quit()                                #
input('Bye.')                                    # Windows

```

深度理解：

- 展示网络编程 + 协议库 (poplib) 的完整流程：连接、认证、stat、list、top (取头)、retr (取全文)、delete (删除)、quit。
- getpass.getpass 不在屏幕回显密码——安全输入的标准做法。
- try/finally 保证无论发生什么都能 server.quit() (释放服务器端邮箱锁) ——异常部分的核心实践。

示例 B-43: SQLite 数据库脚本 (sqldbbase)

```

# Example B-43. Part7/Self-Study-Demos/sqldbbase.py
# people.db SQLite
import sqlite3, time
conn = sqlite3.connect('people.db')          #
curs = conn.cursor()                         # SQL
#
tbl = curs.execute('select name from sqlite_master where name = \'...
if tbl.fetchone() is None:                   #
    print('Making table anew')
    curs.execute('create table people (name, job, pay)') #
    recs = [('Pat', 'mgr', 40000), ('Sue', 'dev', 60000), ('Bob', ...
    for rec in recs:
        curs.execute('insert into people values (?, ?, ?)', rec) ...
    conn.commit()                             #
#
print('Rows:')
curs.execute('select * from people')
for row in curs.fetchall():

```

```

    print(row)
# dev
print('Devs:')
curs.execute("select name, pay from people where job = 'dev'")
colnames = [desc[0] for desc in curs.description]      #
while row := curs.fetchone():                          #
    print('-' * 30)
    for (name, value) in zip(colnames, row):
        print(f'{name:<4} => {value}')                # f-string
# dev
secs = int(time.time())                                # UTC
curs.execute('update people set pay = ? where job = ?', [secs, 'de...
conn.commit()

```

深度理解：

- sqlite3: Python 内置 SQLite 数据库驱动——无需服务器，一个文件就是数据库。连接 (connect) → 游标 (cursor) → execute (SQL) → commit (提交) → fetch (取结果)。
- 参数化查询: execute('insert ... values (?, ?, ?)', rec) 用 ? 占位符传参——防止 SQL 注入的最佳实践。
- sqlitemaster 是 SQLite 的系统表，用来检查表是否存在；curs.description 提供列名。
- while row := curs.fetchone(): 是 Python 3.8+ 的海象运算符 (walrus operator) ——在条件里赋值。
- 综合了大全书的多项技术：异常 (try/finally)、文件、字符串格式化 (%、format、f-string)、推导式、zip、时间戳。

附录总结

技术要点回顾 (Key Takeaways)

· 七大 Part 的技术全景：

· Part I (入门)：REPL vs 脚本、模块导入缓存、字节码与 .pyc、shebang、Traceback 阅读、循环引用与垃圾回收。

· Part II (对象)：索引与切片语义 (越界报错 vs 自动缩放)、不可变性、字典键的可哈希性、文件读写与 seek 回卷。

· Part III (语句)：循环的 else 子句、if/match/字典/列表四种选择写法、list.sort() 返回 None 的陷阱、sorted() 内置。

· Part IV (函数)：多态、args/kwargs/默认值的匹配规则、推导式 vs map vs 循环、生成器的惰性、递归的栈深度限制、计时与基准测试 (timeit、timer2)。

· Part V (模块)：import vs from 的命名空间语义、if name == 'main': 自测惯例、包与 init.py、reload、循环导入的成因与规避。

· Part VI (类)：继承与动态分派 (模板方法)、运算符重载 (add/radd)、getattr 代理与内建操作不拦截的局限、类变量 vs 实例变量、组合 vs 继承、mixin 与类树 (bases)。

- Part VII (异常) : try/except/else、异常类与实例、as 绑定、sys.excinfo 与 traceback、装饰器包装异常、参数化 SQL 与 GUI 事件循环等综合应用。

- 贯穿全书的主线：名字是引用（赋值不复制）——它解释了循环结构、字典共享、类变量共享、import 命名空间等几乎所有"反直觉"现象。

- 性能认知：标准库 C 实现 (math.factorial、math.sqrt) 通常碾压手写 Python 循环；递归最慢且有栈深上限；基准结果随平台/解释器变化，要自己实测。

学习建议 (Learning Advice)

- 重要程度：5/5 星——习题是检验理解的最快途径，本附录是"答案卷"，两者配合使用效果最佳。

- 应该掌握到什么程度：

- 能够不看答案独立完成每题，并解释每一步输出为什么如此；

- 对每个"为什么"（为什么切片越界安全、为什么 sort 返回 None、为什么 getattr 拦不住 +）都能说出底层机制；

- 能把答案中的代码改造成自己的版本（作者明说"任何合理方案都得满分"）——这才是真正的掌握。

- 后续学习路线：

- 巩固：重做各 Part 的 "Test Your Knowledge" 练习，对照本附录逐题订正；

- 深入：阅读第 39 章（装饰器细节）、第 40 章（元类与特殊属性）、第 41 章（编码规范）——本附录的多处内容在那里有更系统的展开；
- 实践：把 Self-Study Demos (largest-.py、summer.py、regtest.py、gui.py、popmail.py、sqldb.py) 跑起来并改造——它们是"标准库 + 真实任务"的最佳示范；
- 拓展：学习 unittest/pytest（把 regtest 的朴素想法升级为正式测试框架）、subprocess（替代 os.popen）、pathlib（替代 os.path 拼接）。